



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

Documentazione

Progetto Ingegneria del Software

Carmine Sgariglia (N86005069)

Mattia Lemma (N86005170)

Massimo Russo (N86005016)

Anno Accademico 2025/2026

Capitolo 1

Introduzione

1.1 Finalità del documento

Questo documento di Specifica dei Requisiti Software fornisce una descrizione completa della piattaforma di gestione bug che verrà sviluppata per l'azienda SoftEngUniNA. Il documento descrive in modo dettagliato i requisiti funzionali e non funzionali della piattaforma che mira a migliorare l'efficienza della gestione collaborativa di issue in progetti software interni all'azienda.

Questa SRS fungerà da base per le successive fasi di progettazione e sviluppo del sistema, garantendo che tutte le parti interessate abbiano una chiara comprensione delle funzionalità e del funzionamento del sistema.

1.2 Descrizione del progetto

1.2.1 Cosa gestisce il software?

Il prodotto software che si vuole realizzare prende il nome di “BugBoard26”, il quale, nella sua prima versione, deve poter svolgere le seguenti funzionalità:

- **Autenticazione:** Gestione l'autenticazione consentendo ad ogni utente di accedere solo alle funzionalità di sua competenza.
- **Creazione di un nuovo utente:** Gli amministratori possono creare nuovi utenti e definirne il ruolo come amministratore o developer.
- **Segnalazione di un issue:** Possibilità di segnalare un nuovo issue con titolo, descrizione e parametri opzionali (contenuti multimediali, categoria, tag, priorità, status). Ogni issue presenta uno stato aggiornato durante la risoluzione, al fine di monitorarne l'avanzamento.

- **Visualizzare issue:** Ricerca degli issue tramite dashboard con filtri ed ordinamento per un accesso rapido ed efficiente.
- **Assegnazione issue:** Gli amministratori possono assegnare gli issue ai developer, con il supporto di un suggerimento automatico fornito dalla piattaforma per la scelta del developer.
- **Cambio stato:** Gli utenti assegnati a un issue possono modificarne lo stato.
- **Gestione cronologia:** Ogni issue dispone di una cronologia che raccoglie i contenuti multimediali e le descrizioni relative ad ogni modifica. (aggiornamento, chiusura, riapertura...).

1.2.2 Cosa non gestisce il software?

Il sistema non gestisce la risoluzione diretta dei bug tramite codice, poiché non prevede il caricamento, l'analisi, la modifica o l'esecuzione di file sorgente. La piattaforma si limita alla segnalazione, assegnazione, tracciamento e documentazione degli issue, senza fornire strumenti di debug o ambienti di sviluppo integrati. In particolare, il software non gestisce:

- L'integrazione con repository di codice sorgente (es. GitHub, GitLab, Bitbucket) per la sincronizzazione automatica degli issue.
- L'esecuzione di script o compilazione del codice per la verifica automatica della risoluzione.
- La comunicazione diretta con ambienti di sviluppo (IDE).
- La gestione delle risorse di progetto (es. tempi, costi, task non legati agli issue).

1.3 Glossario

Questa sezione raccoglie i termini tecnici, acronimi e parole chiave utilizzati all'interno del documento di Specifica dei Requisiti Software per il progetto **BugBoard26**. L'obiettivo del glossario è garantire una **comprensione univoca e coerente** dei concetti impiegati, evitando ambiguità interpretative tra gli attori coinvolti nel processo di analisi, progettazione e sviluppo. Ogni voce è accompagnata da una breve **descrizione esplicativa** che ne chiarisce il significato nel contesto del sistema e ne delimita l'ambito d'applicazione.

Tabella 1.1: Glossario

Termine	Descrizione
Ambiente di sviluppo (IDE)	Software utilizzato dai developer per scrivere, testare e fare debug sul codice (es. IntelliJ, Visual Studio Code). Il sistema non si integra con ambienti di sviluppo esterni.
Amministratore	Utente con privilegi avanzati che può creare nuovi utenti, assegnare <i>issue</i> e gestire i ruoli nel sistema.
API	Insieme di regole e protocolli che permettono la comunicazione tra diversi software. In BugBoard26 consente l'integrazione con servizi esterni.
Autenticazione	Processo di verifica delle credenziali che consente l'accesso alle funzionalità del sistema in base al ruolo dell'utente.
Bug	Malfunzionamento o errore nel codice o nel comportamento del software. In BugBoard26 coincide con il concetto di <i>issue</i> .
BugBoard26	Nome del prodotto software oggetto di questo SRS, piattaforma di gestione collaborativa di <i>issue</i> .
Chat	Sistema che permette lo scambio di messaggi, file e aggiornamenti per la gestione di un <i>issue</i> .
Compilazione	Processo di traduzione del codice sorgente in linguaggio macchina eseguibile. BugBoard26 non include funzionalità di compilazione o verifica automatica del codice.
Cronologia	Registro delle modifiche di un' <i>issue</i> , contenente immagini, descrizioni e azioni effettuate dagli utenti durante il suo ciclo di vita.

Continua nella pagina successiva

Termine	Descrizione
Dashboard	Interfaccia principale per la consultazione e la gestione delle <i>issue</i> , dotata di filtri, ordinamenti e funzioni di ricerca.
Database	Sistema per l'archiviazione e la gestione strutturata dei dati.
Developer	Utente incaricato di lavorare sulle <i>issue</i> assegnate e aggiornare il loro stato e la cronologia.
File sorgente	File contenente il codice di un programma scritto in un linguaggio di programmazione. Il sistema non gestisce il caricamento, l'analisi o l'esecuzione di file sorgente.
Filtri	Strumenti della dashboard che permettono di selezionare e visualizzare solo le <i>issue</i> che soddisfano criteri specifici, come stato, priorità o tipo
Issue	Segnalazione di un problema, errore o attività da risolvere o gestire all'interno di un progetto software.
Media	Foto, pdf, video (contenuto multimediale).
OTP	One-Time Password, è un codice temporaneo monouso usato per verificare l'identità dell'utente. Viene spesso inviato via email o SMS per operazioni come accesso, conferma o recupero password.
Priorità	Livello di urgenza o importanza assegnato a un' <i>issue</i> (es. bassa, media, alta).
Repository	Archivio centralizzato in cui viene memorizzato e versionato il codice sorgente di un progetto software (es. GitHub, GitLab). Il sistema non interagisce direttamente con repository esterni.
Script	Sequenza di comandi o istruzioni eseguite automaticamente da un interprete. Il sistema non consente l'esecuzione di script o automatismi di codice.
Server	Computer o sistema che fornisce servizi, risorse o dati ad altri dispositivi tramite rete.

Continua nella pagina successiva

Termine	Descrizione
Sistema, Piattaforma	Insieme delle componenti software che costituiscono BugBoard26 , permettendo la gestione di utenti e <i>issue</i> tramite interfaccia, logica applicativa e database. Non include strumenti esterni come IDE o repository di codice.
SSE	tecnologia che consente al server di inviare aggiornamenti in tempo reale al client tramite una connessione HTTP aperta.
SoftEngUniNa	Azienda per cui viene sviluppata la piattaforma BugBoard26 .
Stato dell'issue	Indica la fase attuale di un' <i>issue</i> (es. <i>aperto</i> , <i>in lavorazione</i> , <i>completato</i> , <i>riaperto</i>).
Suggerimento automatico	Funzionalità che propone all'amministratore a quale utente assegnare un' <i>issue</i> , basandosi su criteri interni (es. carico di lavoro o competenza).
Tag	Etichetta utilizzata per classificare le <i>issue</i> in base a categoria, tecnologia o contesto.
Team	Insieme di developers e admin che lavorano su un progetto.
Tipo, Categoria	Attributo utilizzato per classificare un' <i>issue</i> in base alla natura del problema o alla tipologia di attività richiesta . Serve a facilitare la ricerca, la gestione e l'assegnazione degli <i>issue</i> all'interno del sistema.
Utente	Qualsiasi persona registrata nel sistema con credenziali di accesso valide. Può essere amministratore o developer .

1.4 Riferimenti

Si è fatto riferimento al documento **IEEE Std. 830-1993 – “IEEE Recommended Practice for Software Requirements Specifications”**, che definisce le linee guida e le buone pratiche per la redazione delle Specifiche dei Requisiti Software.

Tale standard è stato utilizzato come riferimento per strutturare la seguente sezione del documento, assicurando una **descrizione chiara, completa e verificabile** dei requisiti funzionali e non funzionali del sistema. In particolare, l'adozione del modello proposto dallo standard IEEE 830-1993 garantisce:

- Un'**organizzazione coerente** delle sezioni del documento;
- La **tracciabilità dei requisiti**, dalla fase di analisi fino allo sviluppo e alla verifica;
- La **distinzione chiara** tra requisiti funzionali e non funzionali del sistema;
- Una **comunicazione efficace** tra committenti e sviluppatori.

Capitolo 2

Descrizione generale

2.1 Prospettiva del prodotto

BugBoard26 è un prodotto software **indipendente da specifici sistemi hardware**; L'unica dipendenza riguarda il dispositivo su cui il sistema viene eseguito. Il corretto funzionamento di **BugBoard26** richiede la presenza di un **DBMS** per la gestione e la persistenza dei dati. **BugBoard26** è **autonomo nel proprio dominio funzionale** (gestione degli issue), ma **non è tecnicamente stand-alone**, poiché richiede un **ambiente server**, un **database** e una **connessione a servizi esterni tramite API** per operare correttamente. Come già indicato nella sezione introduttiva, **BugBoard26** è eseguibile su un server ed è accessibile tramite interfaccia **web-based**, garantendo semplicità d'uso e indipendenza dall'ambiente hardware.

2.2 Modellazione tramite use cases diagram

Il seguente **Use Case generale** descrive, in forma sintetica, il comportamento complessivo del sistema **BugBoard26** e le principali interazioni tra gli attori e le sue funzionalità. Esso fornisce una **visione d'insieme** delle operazioni che gli utenti possono svolgere all'interno della piattaforma, delineando i flussi principali che caratterizzano la gestione collaborativa degli issue e dei progetti. Questo caso d'uso rappresenta il **livello più alto di astrazione** del sistema e l'obiettivo è illustrare come gli attori (amministratori e developer) interagiscono con il sistema per creare, assegnare, monitorare e risolvere issue in modo strutturato ed efficiente.

2.2.1 Sistema di autenticazione

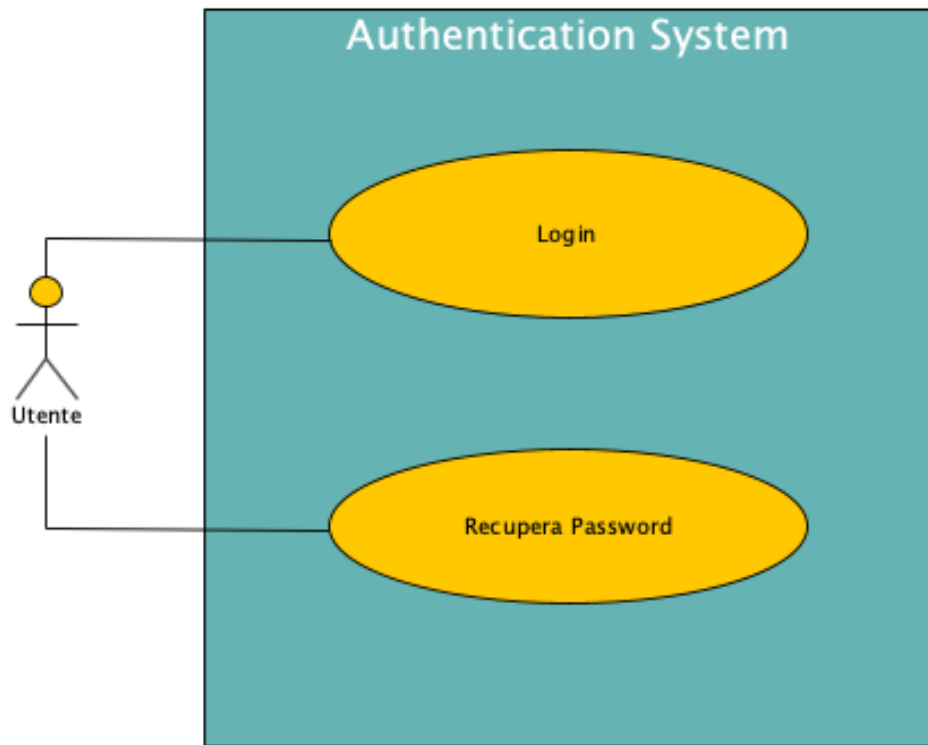


Figura 2.1: Use case diagram sistema di autenticazione

2.2.2 Funzionalità utenti autenticati

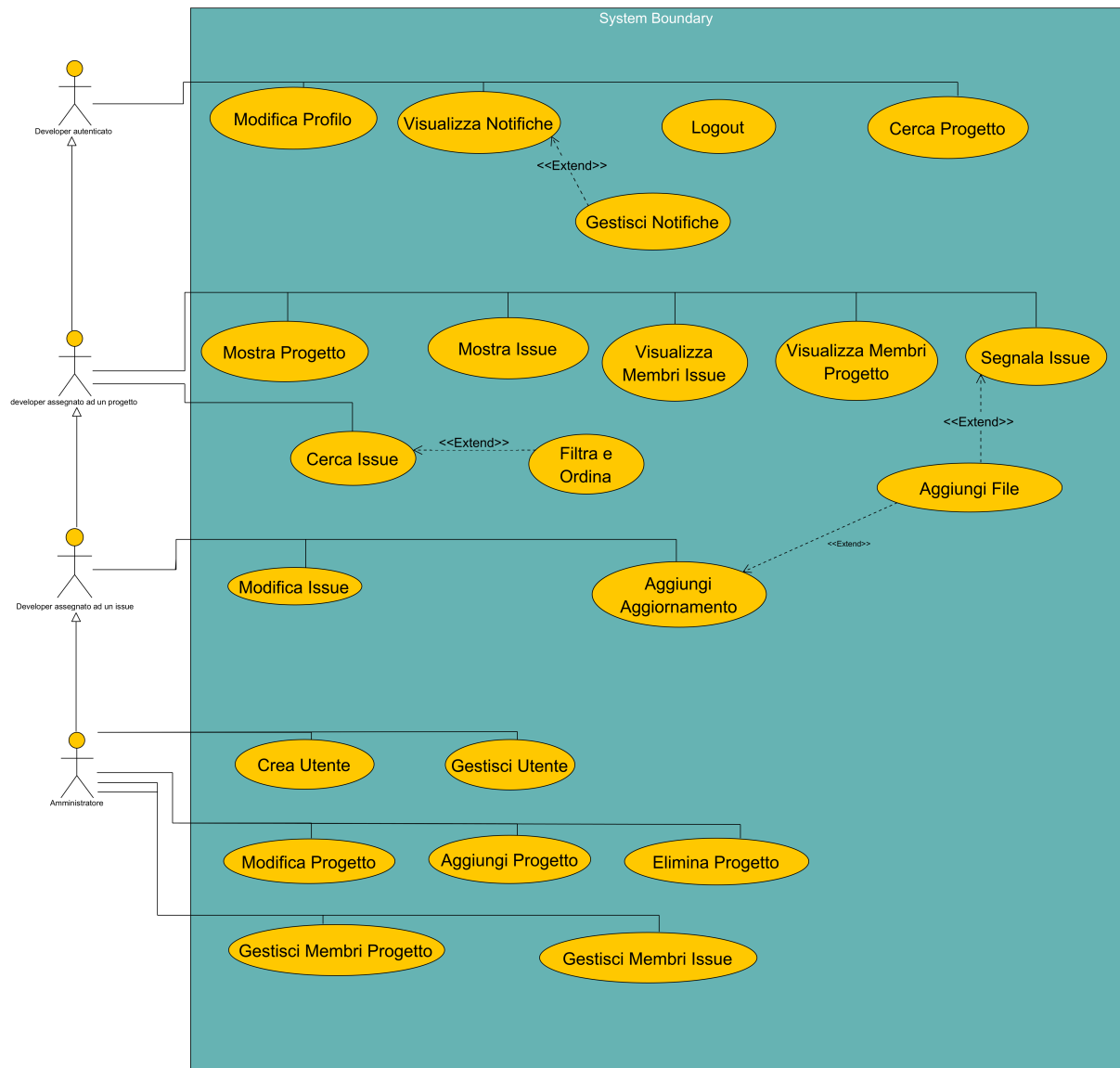


Figura 2.2: Use case diagram funzionalità utenti autenticati

2.3 Funzionalità del prodotto

BugBoard26 deve fornire le funzionalità di seguito riportate che per motivi di chiarezza sono state organizzate in categorie:

- **Gestione account** - Gestisce l'accesso e la creazione degli account.
 - **Autenticazione:** Consente l'accesso al sistema, permettendo a ciascun utente di utilizzare solo le funzionalità del proprio ruolo.
 - **Recupera password:** Consente il recupero della password associata al proprio account.
 - **Logout;** Consente all'utente di uscire dal sistema e tornare alla schermata di login.
 - **Creazione utente:** Permette agli amministratori di creare nuovi profili, specificando se di tipo amministratore o developer.
 - **Modifica profilo:** Consente la modifica dei dati del proprio profilo.
 - **Gestisci utente:** Consente ad un amministratore di gestire tutte le funzionalità di un qualsiasi utente mantenendone la cronologia di azioni.
- **Gestione dei progetti** - Gestire i progetti attivi e aggiungere membri ai team per l'assegnazione degli issue.
 - **Cerca progetto:** Permette di cercare un progetto tramite una barra di ricerca.
 - **Mostra Progetto:** Visualizza tutte le informazioni e gli issue relativi ad un progetto.
 - **Aggiungi progetto:** Consente a un amministratore di creare un nuovo progetto.
 - **Elimina progetto:** Consente a un amministratore di rimuovere un progetto esistente.
 - **Modifica progetto:** Consente a un amministratore di aggiornare le informazioni relative a un progetto.
 - **Gestisci membri:** Consente all'amministratore di gestire un developer assegnandogli o rimuovendolo da un progetto, così da potergli eventualmente assegnare gli issue corrispondenti.
 - **Visualizza membri:** Consente agli amministratori e ai developer di visualizzare la lista dei membri assegnati ad un progetto,
- **Gestione issue** - Segnalare, assegnare e aggiornare i problemi nei progetti

- **Mostra issue:** Visualizza un issue individuato all'interno di un progetto.
- **Modifica issue:** Consente la modifica di alcuni campi di un issue.
- **Segnala issue:** Consente di creare una nuova segnalazione per riportare un problema riscontrato all'interno di un progetto.
- **Gestisci membri issue:** Indica la possibilità dell'amministratore di assegnare o rimuovere dall'issue un utente.
- **Aggiungi aggiornamento:** Consente di inserire un commento multimediale per descrivere l'avanzamento o le attività svolte su un issue.
- **Aggiungi file:** Consente di allegare un file (es. .jpeg) a un aggiornamento di un issue, funzionalità collegata alle opzioni Aggiungi aggiornamento e Segnala issue.
- **Cerca issue:** Permette di cercare gli issue attraverso delle parole chiave.
- **Filtra e ordina risultati:** Consente di visualizzare nella dashboard solo gli issue filtrati.
- **Visualizza membri issue:** Consente agli amministratori e ai developer di visualizzare la lista dei membri assegnati ad un issue,
- **Gestione delle notifiche** - Gestisce la visualizzazione e l'interazione con le notifiche.
 - **Visualizza notifiche:** Consente all'utente di visualizzare un elenco di notifiche relative ad eventi di suo interesse.
 - **Gestisci notifiche:** Consente all'utente di cliccare, leggere ed eliminare le notifiche.

2.4 Caratteristiche degli utenti

Gli utenti di **BugBoard26** devono possedere una **conoscenza di base dell'utilizzo del computer e dei principali software applicativi**, necessaria per interagire con il pannello di controllo, progettato per essere **semplice e intuitivo**. Tuttavia, è richiesta una **preparazione più approfondita** per le attività di gestione avanzata, come l'assegnazione corretta degli issue ai developer appropriati, la risoluzione dei bug, la redazione di commenti tecnici pertinenti e la **collaborazione efficace all'interno del team di sviluppo**. Tali competenze garantiscono una comunicazione chiara e la registrazione di **dati coerenti e completi nella cronologia** del sistema. Il sistema prevede due tipologie principali di utenti:

- **Amministratore:** Il ruolo admin estende quello del developer, includendo tutte le sue azioni e operazioni. Inoltre può creare nuovi account developer e amministratore, creare e modificare nuovi progetti, assegnare i developer ai progetti ed assegnare gli issue ai developer. È inoltre responsabile del corretto funzionamento generale e dell'organizzazione delle attività sulla piattaforma.
- **Developer:** Può segnalare nuovi issue, visualizzare quelli a lui assegnati, modificarne lo stato e aggiungere immagini e descrizioni in caso di avanzamento o completamento del lavoro, contribuendo così alla cronologia dell'issue e alla tracciabilità delle attività svolte.

Capitolo 3

Requisiti specifici

3.1 Requisiti funzionali

Tabella 3.1: Requisiti funzionali – Autenticazione

Campo	Dettagli
Funzionalità	Autenticazione
Utenti	Amministratore, Developer
Introduzione	All'avvio del sistema, l'utente deve effettuare l'autenticazione per accedere al proprio profilo e alle relative funzionalità.
Input	• Email: identificativo personale dell'utente (<i>obbligatorio</i>). • Password: chiave di accesso personale (<i>obbligatoria</i>).
Elaborazione	Il sistema verifica la correttezza delle credenziali inserite e, in caso di esito positivo, abilita le funzionalità corrispondenti al ruolo dell'utente autenticato.
Output	• Messaggio di errore in caso di autenticazione non riuscita. • Messaggio di benvenuto e accesso alla piattaforma in caso di autenticazione riuscita.

Tabella 3.3: Requisiti funzionali – Recupera password

Campo	Dettagli
Funzionalità	Recupera password
Utenti	Amministratore, Developer
Introduzione	Consente il recupero della password associata all'account dell'utente.
Input	- Click sull'apposito pulsante Recupera password. - Digitazione dell'indirizzo email associato all'account da recuperare (<i>obbligatoria</i>).
Elaborazione	Il sistema registra i dati forniti, verifica l'esistenza di un account associato a tale indirizzo email e, in caso affermativo, invia un'email contenente le istruzioni e i passaggi da seguire per la modifica della password.
Output	- Messaggio di conferma in caso di richiesta inviata con successo. - Messaggio di errore se l'<i>email</i> non è associata ad alcun account. - Ritorno alla schermata di login in caso di annullamento dell'operazione.

Tabella 3.5: Requisiti funzionali – Logout

Campo	Dettagli
Funzionalità	Logout
Utenti	Amministratore, Developer
Introduzione	Consente all'utente di uscire dalla piattaforma e di ritornare sulla pagina del login.
Input	- Click sull'icona del avatar in alto a destra. - Click sull'apposito tasto logout.
Elaborazione	Il sistema elimina l'autenticazione dell'utente e lo reindirizza sulla pagina di login pronto ad effettuare un nuovo accesso
Output	Reindirizzamento sulla pagina di login.

Tabella 3.7: Requisiti funzionali – Creazione profilo

Campo	Dettagli
Funzionalità	Creazione utente
Utenti	Amministratore
Introduzione	La funzionalità consente agli amministratori di creare nuovi profili utente, assegnando loro un ruolo specifico all'interno del sistema.
Input	• Nome: nome dell'utente (<i>obbligatorio</i>). • Cognome: cognome dell'utente (<i>obbligatorio</i>). • Email: indirizzo email personale usato per l'accesso e la comunicazione con l'utente. (<i>obbligatorio</i>). • Ruolo: tipo di profilo da assegnare (<i>Amministratore</i> o <i>Developer</i>) (<i>obbligatorio</i>).
Elaborazione	Il sistema registra i dati inseriti e crea un nuovo account con le informazioni e i permessi specificati dall'amministratore.
Output	• Invio tramite mail all'utente di nome utente e password temporanea per il primo accesso all'account. • Messaggio di errore in caso di dati mancanti o già associati a un altro utente.

Tabella 3.9: Requisiti funzionali – Modifica profilo

Campo	Dettagli
Funzionalità	Modifica profilo
Utenti	Amministratore, Developer
Introduzione	La funzionalità consente a qualsiasi utente di modificare i dati del proprio profilo.
Input	• Nome: nome dell'utente (<i>obbligatorio</i>). • Cognome: cognome dell'utente (<i>obbligatorio</i>). • Username: nome univoco del profilo (<i>obbligatorio</i>). • Email: email univoca del profilo (<i>obbligatoria</i>). • Password attuale: chiave di accesso personale attuale (<i>obbligatoria</i>). • Password: nuova chiave di accesso personale (<i>obbligatoria</i>).
Elaborazione	Il sistema registra i dati inseriti e li assegna all'utente se essi non presentano errori o conflitti di unicità. se avviene il cambio password il sistema verifica che la precedente sia coerente.
Output	• Messaggio di conferma in caso di modifica avvenuta con successo. • Messaggio di errore in caso di errato inserimento in un campo. • Messaggio di errore in caso di username o email già assegnato a un altro profilo.

Tabella 3.11: Requisiti funzionali – Gestisci utente

Campo	Dettagli
Funzionalità	Gestisci utente
Utenti	Amministratore
Introduzione	La funzionalità consente all'amministratore di gestire un utente sul sistema, indipendentemente dal suo ruolo (<i>amministratore</i> o <i>developer</i>).
Input	• Selezione utente: scelta dell'utente da gestire tramite ricerca o elenco. • Username: Click sul tasto "Save changes" per confermare l'operazione (<i>obbligatorio</i>).
Elaborazione	Il sistema verifica il corretto inserimento di tutti i dati nel form del sistema; in caso positivo procede con le modifiche dell'account. L'amministratore può comunque annullare l'operazione prima della conferma definitiva.
Output	• Messaggio di errore in caso di dati non validi o non correttamente inseriti. • Messaggio di conferma in caso di operazione avvenuta con successo. • Possibilità di annullare l'azione prima della conferma.

Tabella 3.13: Requisiti funzionali – Cerca progetto

Campo	Dettagli
Funzionalità	Cerca progetto
Utenti	Amministratore, Developer
Introduzione	La funzionalità consente a qualsiasi developer o amministratore di cercare un determinato progetto al fine di velocizzarne l'accesso.
Input	• Titolo di un progetto inserito nella barra di ricerca.
Elaborazione	Il sistema elabora la stringa di testo inserita e verifica se ci sono corrispondenze tra il testo digitato e il titolo di un progetto visualizzabile.
Output	• Messaggio di errore se il progetto non esiste. • Se il progetto esiste, stampa a schermo la <i>card</i> del progetto.

Tabella 3.15: Requisiti funzionali – Mostra progetto

Campo	Dettagli
Funzionalità	Mostra progetto
Utenti	Amministratore, Developer
Introduzione	La funzionalità consente all'utente di selezionare e accedere a un progetto specifico per visualizzare e gestire gli issue e le relative impostazioni di pertinenza.
Input	Accesso al progetto desiderato tramite un click sul suo nome nella lista dei progetti disponibili.
Elaborazione	Il sistema verifica i permessi dell'utente, mostrando a schermo le informazioni pertinenti.
Output	Apertura dell'ambiente di lavoro del progetto selezionato con conseguente caricamento degli issue e delle relative informazioni.

Tabella 3.17: Requisiti funzionali – Aggiungi progetto

Campo	Dettagli
Funzionalità	Aggiungi progetto
Utenti	Amministratore
Introduzione	La funzionalità consente all'amministratore di creare un nuovo progetto e di aggiungere i membri che vi parteciperanno.
Input	• Nome progetto: titolo identificativo del nuovo progetto (<i>obbligatorio</i>). • Descrizione: breve testo descrittivo del progetto (<i>obbligatoria</i>). • Icona progetto: icona del progetto tra quelle disponibili di default (<i>obbligatoria</i>). • Colore progetto: colore della cartella del progetto (<i>obbligatoria</i>). • Utenti partecipanti: elenco degli utenti da aggiungere al progetto (<i>facoltativo</i>).
Elaborazione	Il sistema registra i dati forniti, crea il nuovo progetto, associa gli utenti selezionati e invia una notifica a ciascun utente aggiunto per informarlo dell'assegnazione al nuovo progetto.
Output	• Messaggio di conferma in caso di creazione e assegnazione riuscite. • Messaggio di errore se i dati inseriti non sono validi.

Tabella 3.19: Requisiti funzionali – Elimina progetto

Campo	Dettagli
Funzionalità	Elimina progetto
Utenti	Amministratore
Introduzione	La funzionalità consente all'amministratore di eliminare definitivamente un progetto esistente dal sistema, rimuovendo tutti i dati e gli <i>issue</i> ad esso associati.
Input	Codice di conferma randomico a 6 cifre da copiare per confermare l'eliminazione del progetto.
Elaborazione	Il sistema verifica la conferma dell'amministratore e, in caso di esito positivo, procede con la rimozione del progetto e di tutte le informazioni correlate, inviando una notifica a tutti i membri del team per informarli dell'eliminazione.
Output	- Messaggio di conferma in caso di eliminazione avvenuta con successo. - Possibilità di annullare l'operazione prima della conferma.

Tabella 3.21: Requisiti funzionali – Modifica progetto

Campo	Dettagli
Funzionalità	Modifica progetto
Utenti	Amministratore
Introduzione	La funzionalità consente all'amministratore di modificare le informazioni di un progetto esistente, modificandone i dettagli.
Input	Nuovo nome progetto: identificativo del progetto da modificare (<i>obbligatorio</i>). Nuova descrizione: descrizione aggiornata del progetto (<i>obbligatorio</i>). nuovo colore progetto: colore della cartella del progetto aggiornata (<i>obbligatoria</i>). nuova icona progetto: icona del progetto aggiornata tra quelle disponibili di default (<i>obbligatorio</i>).
Elaborazione	Il sistema aggiorna le informazioni fornite, mantenendo la coerenza dei dati collegati agli <i>issue</i> .
Output	Messaggio di conferma di avvenuta modifica del progetto.

Tabella 3.23: Requisiti funzionali – Gestisci membri

Campo	Dettagli
Funzionalità	Gestisci membri
Utenti	Amministratore
Introduzione	La funzionalità consente all'amministratore di gestire un developer per assegnarlo o rimuoverlo dal progetto e permettere l'assegnazione e la risoluzione degli <i>issue</i> .
Input	• Username: Scelta del developer da aggiungere da una lista di sistema ordinata per occupazione dei developer (<i>obbligatorio</i>).
Elaborazione	Il sistema associa il developer al progetto selezionato e invia una notifica al nuovo membro per informarlo che è stato aggiunto al progetto. Se viene rimosso il sistema lo rimuove dal progetto ed invia una notifica di rimozione dal progetto al membro interessato.
Output	Messaggio di notifica in caso di aggiunta o rimozione avvenuta con successo.

Tabella 3.25: Requisiti funzionali – Visualizza membri

Campo	Dettagli
Funzionalità	Visualizza membri
Utenti	Amministratore e developer
Introduzione	La funzionalità consente all'amministratore e al developer di visualizzare la lista di tutti gli utenti assegnati ad un progetto.
Input	• Click sull'icona della matita nella pagina del progetto.
Elaborazione	Il sistema al click del tasto con l'icona della matita apre una lista che contiene tutti gli utenti assegnati al progetto.
Output	Lista con gli utenti assegnati al progetto.

Tabella 3.27: Requisiti funzionali – Mostra issue

Campo	Dettagli
Funzionalità	Mostra issue
Utenti	Amministratore, Developer
Introduzione	Rappresenta una segnalazione individuata all'interno di un progetto.
Input	• Uno specifico <i>issue</i> individuato all'interno di un progetto.
Elaborazione	Il sistema elabora le informazioni da mostrare relative all' <i>issue</i> .
Output	Informazioni dell'issue: • Titolo. • Descrizione. • Eventi degli issue. • Categoria. • Tag. • Livello di importanza. • Data inizio. • Stato. • Developer assegnati all'<i>issue</i>.

Tabella 3.29: Requisiti funzionali – Modifica issue

Campo	Dettagli
Funzionalità	Modifica issue
Utenti	Amministratori o Developer assegnati all'issue
Introduzione	Rappresenta la possibilità di modificare uno dei seguenti attributi di un <i>issue</i> .
Input	• Titolo issue: nome identificativo dell'<i>issue</i> (<i>obbligatorio</i>). • Livello di importanza: priorità o gravità del problema (<i>obbligatorio</i>). • Categoria: categoria dell'issue. (<i>obbligatorio</i>). • Stato: stato dell'issue. (<i>obbligatorio</i>). • Tag: tag dell'issue. (<i>facoltativo</i>).
Elaborazione	Il sistema registra e aggiorna le informazioni dell' <i>issue</i> in base ai dati forniti.
Output	• Messaggio di conferma di modifica issue avvenuta con successo

Tabella 3.31: Requisiti funzionali – Segnala issue

Campo	Dettagli
Funzionalità	Segnala issue
Utenti	Amministratore, Developer
Introduzione	Rappresenta una segnalazione individuata all'interno di un progetto e può essere creata da un qualsiasi utente che opera su tale progetto.
Input	• Titolo issue: nome identificativo dell'<i>issue</i> (<i>obbligatorio</i>). • Descrizione: dettagli del problema (<i>obbligatorio</i>). • Livello di importanza: priorità o gravità del problema (<i>obbligatorio</i>). • File: eventuale file da allegare (<i>facoltativo</i>). • Categoria: categoria dell'issue. (<i>obbligatorio</i>). • Stato: stato dell'issue. (<i>obbligatorio</i>). • Tag: tag dell'issue. (<i>facoltativo</i>).
Elaborazione	Il sistema registra le informazioni dell' <i>issue</i> , la collega al progetto selezionato e invia una notifica agli amministratori con ricezione della notifica attiva.
Output	• Messaggio di conferma in caso di creazione riuscita. • Messaggio di errore se i dati sono incompleti o non validi.

Tabella 3.33: Requisiti funzionali – Gestisci membri issue

Campo	Dettagli
Funzionalità	Gestisci membri issue
Utenti	Amministratore
Introduzione	Consente all'amministratore di assegnare o rimuovere uno o più developer ad un <i>issue</i> , eventualmente con il supporto del suggerimento automatico del sistema.
Input	L'amministratore seleziona l'<i>issue</i> a cui vuole assegnare o rimuovere un developer tramite una lista ricercabile, individua il developer sui cui effettuare questa modifica. La lista è ordinata in base ai suggerimenti del sistema: i developer più adatti compaiono nelle prime posizioni, tutti i developer sono contrassegnati da un indice numerico che indica il numero di issue a cui sono assegnati.
Elaborazione	Il sistema assegna o rimuove il developer selezionato dall'issue, aggiorna i dati relativi alle assegnazioni e invia una notifica al developer per informarlo del suo nuovo stato.
Output	Notifica di conferma in caso di assegnazione o rimozione riuscita.

Tabella 3.35: Requisiti funzionali – Aggiungi aggiornamento

Campo	Dettagli
Funzionalità	Aggiungi aggiornamento
Utenti	Amministratori o Developer assegnati all' <i>issue</i>
Introduzione	Consente di aggiungere una nota multimediale o descrittiva che documenta l'avanzamento del lavoro su un <i>issue</i> .
Input	Testo aggiornamento: descrizione delle attività svolte (<i>obbligatorio</i>). File allegato: immagine o documento facoltativo (<i>facoltativo</i>).
Elaborazione	Il sistema registra l'aggiornamento e lo collega alla cronologia dell' <i>issue</i> , rendendolo visibile a tutti gli utenti autorizzati.
Output	Inserimento di un nuovo aggiornamento nella dashboard dell'<i>issue</i>.

Tabella 3.37: Requisiti funzionali – Aggiungi file

Campo	Dettagli
Funzionalità	Aggiungi file
Utenti	Amministratori o Developer assegnati all' <i>issue</i>
Introduzione	Consente di allegare più di un file (es. <i>.jpeg</i> , <i>.pdf</i>) alla creazione o all'aggiornamento di un <i>issue</i> , legato alle funzionalità <i>Aggiungi aggiornamento</i> e <i>Crea nuovo issue</i> .
Input	• File: documento o immagine da allegare (<i>obbligatorio</i>).
Elaborazione	Il sistema carica e associa il file all'aggiornamento selezionato, rendendolo accessibile tramite la cronologia dell' <i>issue</i> .
Output	• Caricamento del nuovo file. • Messaggio di errore se il file non è valido o supera le dimensioni consentite.

Tabella 3.39: Requisiti funzionali – Cerca issue

Campo	Dettagli
Funzionalità	Cerca issue
Utenti	Amministratore, Developer
Introduzione	La funzionalità consente a qualsiasi utente in un progetto di cercare un determinato <i>issue</i> al fine di velocizzarne l'accesso.
Input	• Qualsiasi titolo di un <i>issue</i> inserito nella barra di ricerca.
Elaborazione	Il sistema elabora la stringa di testo inserita e verifica se ci sono corrispondenze tra il testo digitato e il titolo di un <i>issue</i> .
Output	• Messaggio di errore se l'<i>issue</i> non esiste. • Se l'<i>issue</i> esiste, stampa a schermo l'<i>issue</i>.

Tabella 3.41: Requisiti funzionali – Filtra ed ordina risultati

Campo	Dettagli
Funzionalità	Filtra ed ordina risultati
Utenti	Amministratore, Developer
Introduzione	Consente di visualizzare nella dashboard solo gli <i>issue</i> filtrati per stato, priorità, tipo o ordinati per data.
Input	Selezionare la tipologia di filtro ed ordine desiderata dal toogle presenti nella dashboard.
Elaborazione	Il sistema filtra gli <i>issue</i> in base ai filtri e all'ordinamento impostato e aggiorna la dashboard con i risultati corrispondenti.
Output	Elenco degli <i>issue</i> filtrati ed ordinati.

Tabella 3.43: Requisiti funzionali – Visualizza membri issue

Campo	Dettagli
Funzionalità	Visualizza membri issue
Utenti	Amministratore e developer
Introduzione	La funzionalità consente all'amministratore e al developer di visualizzare la lista di tutti gli utenti assegnati ad un issue.
Input	Click sull'icona della matita nella pagina dell'issue.
Elaborazione	Il sistema al click del tasto con l'icona della matita apre una lista che contiene tutti gli utenti assegnati all'issue.
Output	Lista con gli utenti assegnati all'issue.

Tabella 3.45: Requisiti funzionali – Visualizza notifiche

Campo	Dettagli
Funzionalità	Visualizza notifiche
Utenti	Amministratore, Developer
Introduzione	La funzionalità consente a ogni utente di visualizzare un elenco cronologico delle notifiche generate dal sistema in seguito ad azioni pertinenti al suo ruolo o alle sue attività.
Input	Accesso all'area notifiche tramite un'icona.
Elaborazione	Il sistema recupera e mostra l'elenco delle notifiche non lette e lette relative all'utente, ordinandole dalla più recente alla meno recente.
Output	• Elenco formattato delle notifiche. • Possibilità per l'utente di cliccare su una notifica per essere reindirizzato alla pagina dell'<i>issue</i> o del progetto corrispondente. • Indicatore visivo per le notifiche non lette.

Tabella 3.47: Requisiti funzionali – Gestisci notifiche

Campo	Dettagli
Funzionalità	Gestisci notifiche
Utenti	Amministratore, Developer
Introduzione	La funzionalità consente a ogni utente di leggere, eliminare e selezionare le notifiche per interagirvi ed essere reindirizzato al punto di interesse associato.
Input	• Accesso all'area notifiche tramite un'icona. • Clic sulla notifica per il reindirizzamento oppure utilizzo dei comandi di eliminazione o marcatura come letta.
Elaborazione	Il sistema recupera e mostra l'elenco delle notifiche lette e non lette relative all'utente, ordinandole dalla più recente alla meno recente, e ne consente l'interazione per il reindirizzamento, la marcatura come letta e l'eliminazione.
Output	• Elenco formattato delle notifiche. • Possibilità per l'utente di fare clic su una notifica per essere reindirizzato alla pagina dell'<i>issue</i> o del progetto corrispondente. • Indicatore visivo per le notifiche non lette. • Possibilità per l'utente di eliminare la notifica o segnalarla come letta.

3.2 Requisiti non funzionali

3.2.1 Interfaccia utente

Il sistema presenta un'interfaccia **semplice, intuitiva e coerente**, accessibile anche a utenti con competenze informatiche di base. Le funzionalità principali devono essere raggiungibili in pochi passaggi e fornite di messaggi chiari in caso di errore o conferma. La dashboard e le sezioni di gestione mantengono una **coerenza grafica e terminologica** in tutto il sistema

3.2.2 Interfaccia hardware

BugBoard26 è una piattaforma **web-based** e non richiede l'utilizzo di dispositivi hardware esterni alla normale configurazione di sistema. Per il corretto funzionamento è necessario disporre di:

- Un computer o dispositivo con browser web compatibile (es. Google Chrome, Mozilla Firefox, Microsoft Edge).
- Connessione Internet stabile, indispensabile per accedere al sito e interagire con le funzionalità del sistema.

3.2.3 Requisiti di performance

BugBoard26 non dà alcuna garanzia riguardante i tempi d'esecuzione delle proprie funzionalità e del corretto funzionamento, di calo delle performance o di perdita di dati dovuti a malfunzionamenti dei dispositivi Hardware, del Sistema Operativo, d'eventuali altri Software, virus e worm informatici o intrusioni nel sistema.

3.2.4 Controllo accessi

L'accesso alle funzionalità è regolato tramite **autenticazione** e **gestione dei ruoli**, assicurando che ogni utente possa operare solo nelle aree di propria competenza. Le credenziali sono conservate in modo sicuro e non sono visualizzabili in chiaro. Solo gli amministratori possono creare e modificare utenti e progetti.

3.2.5 Sicurezza dei file

I file caricati dagli utenti sono salvati in modo sicuro e accessibili solo da chi dispone delle relative autorizzazioni. Il sistema limita la tipologia e la dimensione dei file caricabili per evitare rischi o sovraccarichi.

3.2.6 Manutenibilità

Il sistema software è sviluppato con paradigma object-oriented al fine di migliorarne la manutenibilità.

3.3 Altri requisiti

3.3.1 Database

Come già indicato nella sezione dedicata ai **requisiti software**, per poter funzionare **BugBoard26** necessita di un **database relazionale**.

Per motivi di ordine pratico, parte della struttura del database può essere **creata e configurata esternamente** al sistema durante la fase di installazione o deploy.

In particolare, alcune tabelle di base (come i dati del primo amministratore) devono essere **popolate da un tecnico o amministratore di sistema** prima dell'avvio operativo della piattaforma.

Capitolo 4

Tabella di Cockburn

Il **diagramma di Cockburn** è una **descrizione strutturata dei casi d'uso** di un sistema. Definisce come gli **attori** interagiscono con il sistema per raggiungere un obiettivo, includendo precondizioni, flusso principale e possibili alternative. Viene **utilizzato per documentare in modo chiaro e completo** il comportamento funzionale del progetto. Di seguito è riportato il diagramma per una funzionalità.

4.0.1 Cerca Issue

USE CASE #1	Segnalare un issue		
Goal in Context	Un utente vuole segnalare un issue		
Preconditions	L'utente deve essere assegnato al progetto in cui vuole segnalare l'issue.		
Success End Condition	Il sistema crea una nuova issue e assegna l'utente all'issue stessa.		
Failed End Condition	L'issue non viene creata e viene mostrato un messaggio di errore all'utente che ha tentato di crearla.		
Primary Actor			
Trigger	Amministratore, o Developer assegnato al progetto.		
Main Scenario	Step n.	Actor 1	System
	1	Click su "Report New Issue" (Schermata M1)	
	2		Richiede inserimento dei dati dell'issue (Schermata M2)
	3	Inserimento dei dati e conferma della segnalazione.	
	4		Torna a M1 e mostra un messaggio di successo. (Schermata M3)
Extension A: "L'utente annulla l'operazione"	7a	Click sul pulsante "cancel"	
	8a		(Schermata M1)
Extension B: "L'utente non compila uno o più campi"	7b	L'utente inserisce valori errati o nulli	
	8b		(Schermata M4)

Figura 4.1: Segnala Issue

Capitolo 5

Mockup (wireframe)

Le seguenti immagini illustrano i **mockup** delle interfacce principali, utili a rappresentare la struttura e l'esperienza d'uso previste dal sistema.

5.1 Autenticazione

Rappresenta la prima interfaccia visualizzata all'avvio dell'applicazione. Consente l'autenticazione dell'utente per permetterne l'accesso all'area principale del sistema. È inoltre possibile recuperare la password associata al profilo in caso di smarrimento.

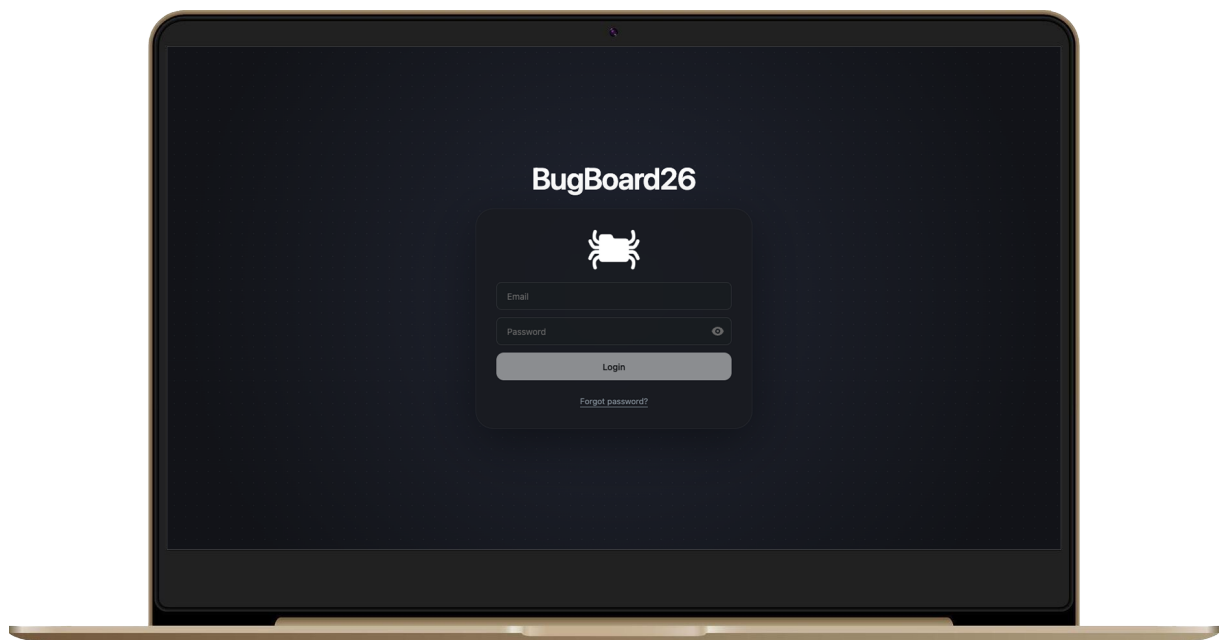


Figura 5.1: Interfaccia per l'Autenticazione

5.1.1 Home di un admin

Rappresenta la Home del software BugBoard26 vista dal lato amministrativo. In questa sezione, l'amministratore può visualizzare e gestire i progetti esistenti. A differenza dell'interfaccia destinata ai developer, è presente la cartella contrassegnata dall'icona "+", che consente l'aggiunta di nuovi progetti.

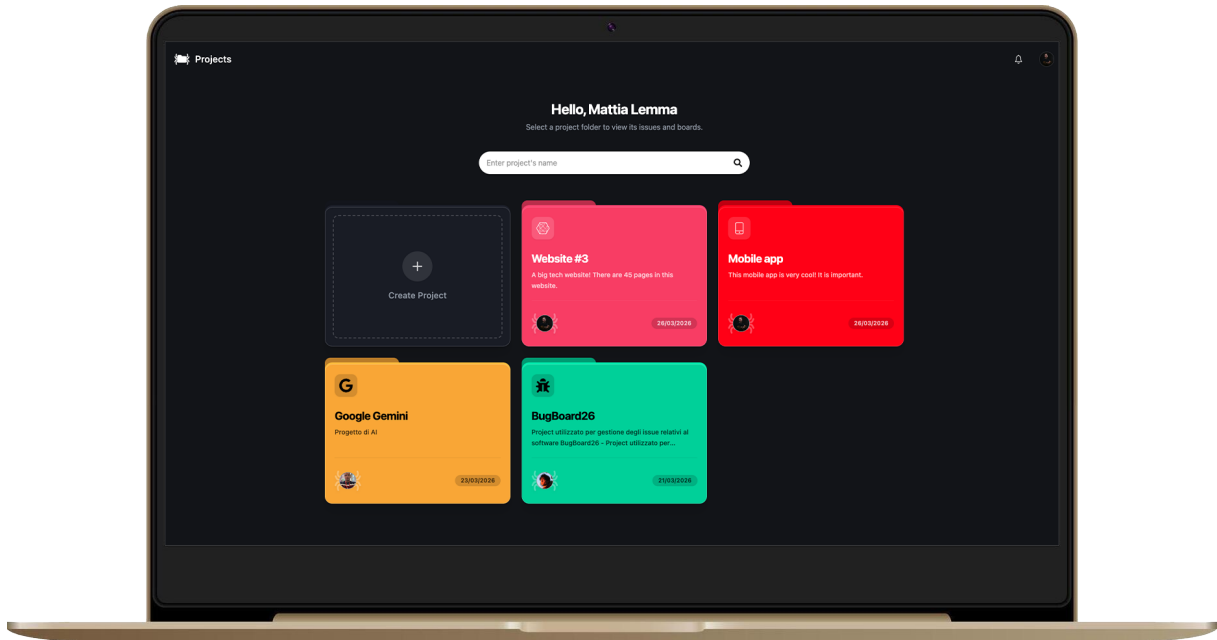


Figura 5.2: Interfaccia Home di un admin

5.1.2 Progetto generico

L'interfaccia consente una rapida visualizzazione di tutti gli issue associati a un progetto. È rappresentata dal punto di vista di un amministratore, il quale, oltre a poter svolgere le stesse operazioni disponibili per un developer (segnalare, gestire e ricercare issue), ha anche la possibilità di gestire il progetto e inserire nuovi utenti al suo interno.

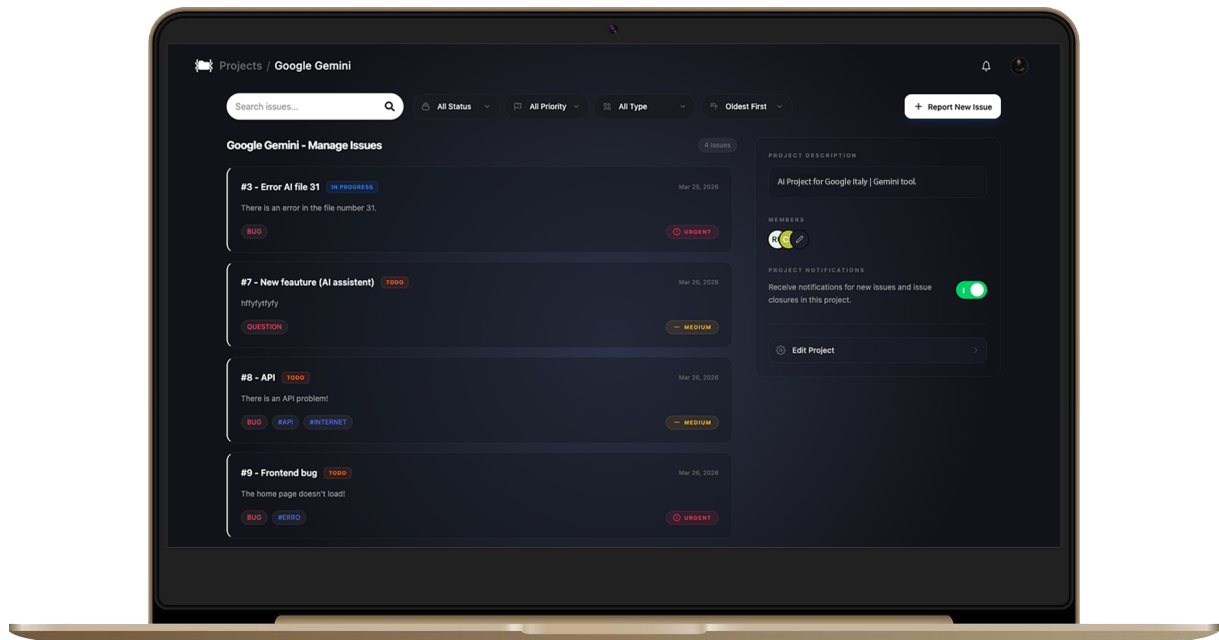


Figura 5.3: Interfaccia generale di un progetto (M1).

5.1.3 Segnalazione issue

La funzionalità Segnala issue consente l'inserimento dei dati necessari all'identificazione di un nuovo issue. È possibile selezionare la categoria, priorità e lo status tramite l'apposito menù a tendina, è inoltre possibile aggiungere i tag inserendo il testo desiderato e confermando l'operazione. Non è possibile superare il limite di caratteri previsto per ogni sezione, per garantire la corretta validazione dei dati durante l'inserimento.

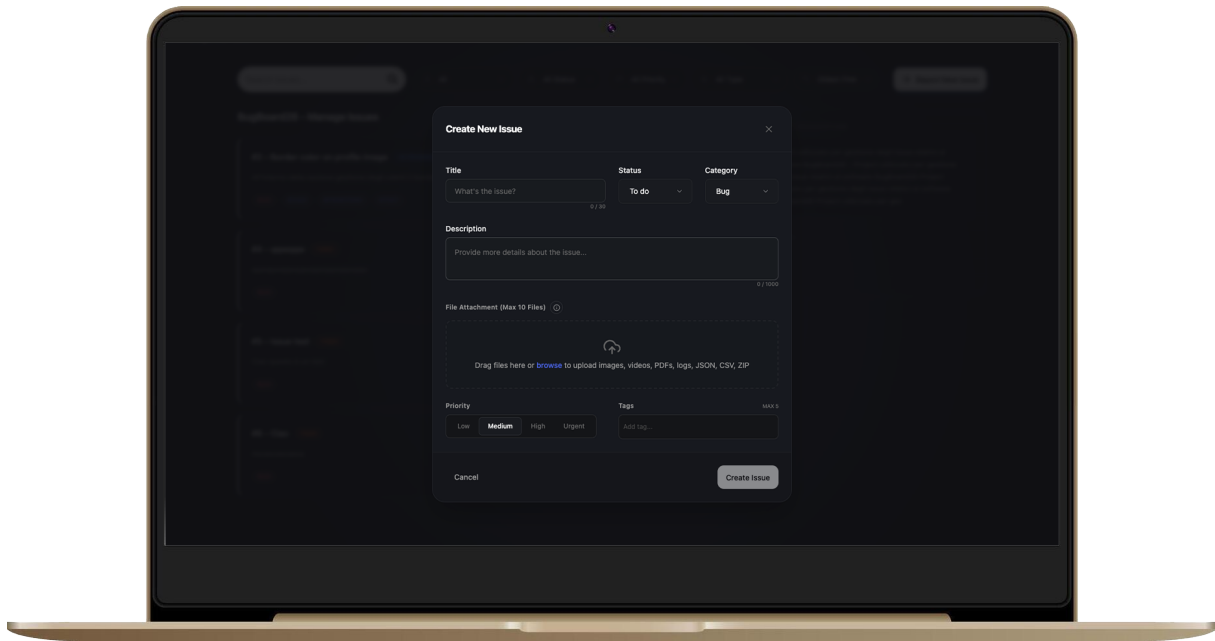
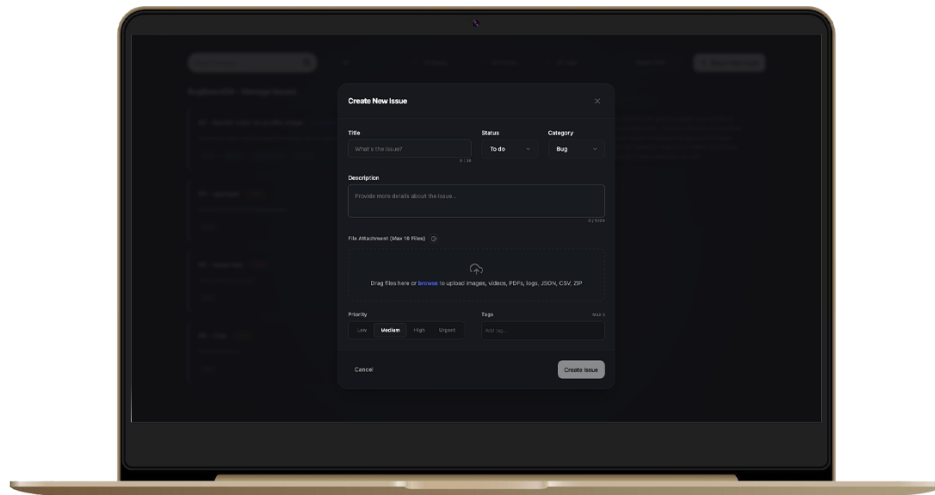


Figura 5.4: Interfaccia per la segnalazione di un issue (M2).

5.1.4 Output per la creazione di un issue

Sono mostrate le finestre di notifica che compaiono durante la creazione di un issue:

1. Errore quando la dimensione del file allegato supera il limite consentito;
2. Errore quando non tutti i campi obbligatori sono stati compilati;
3. Conferma in caso di inserimento avvenuto con successo.



POSSIBILI OUTPUT

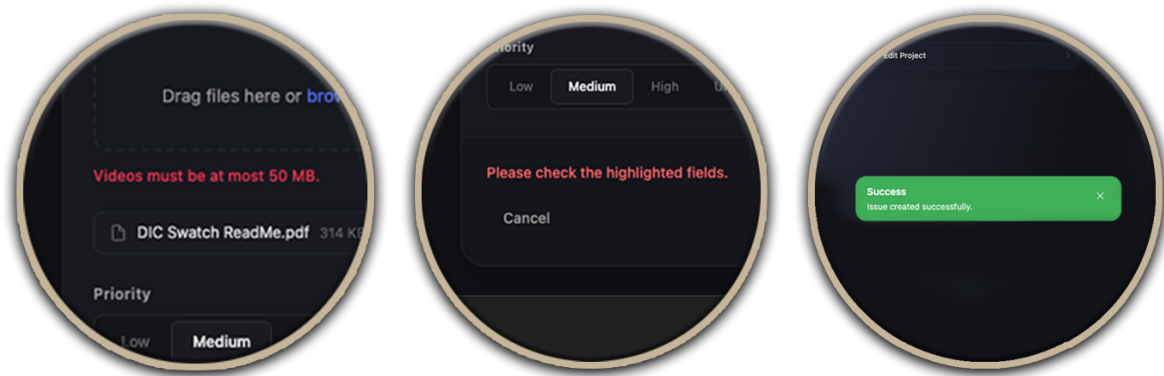


Figura 5.5: Possibili messaggi di errore o conferma durante la creazione di un issue.

Rispettivamente: (M5), (M4), (M3)

Capitolo 6

Architettura del Sistema

Il sistema è progettato secondo un'architettura **Three-Tier** (a tre livelli) basata sul paradigma **Client-Server**, con una chiara separazione delle responsabilità tra i diversi strati applicativi. Tale scelta consente di organizzare il sistema in componenti logicamente indipendenti ma strettamente cooperanti, migliorando la comprensibilità del progetto, la manutenibilità del codice e la possibilità di evolvere i singoli livelli in modo controllato. In particolare, l'architettura adottata da **BugBoard26** riflette una suddivisione netta tra interfaccia utente, logica applicativa e persistenza dei dati.

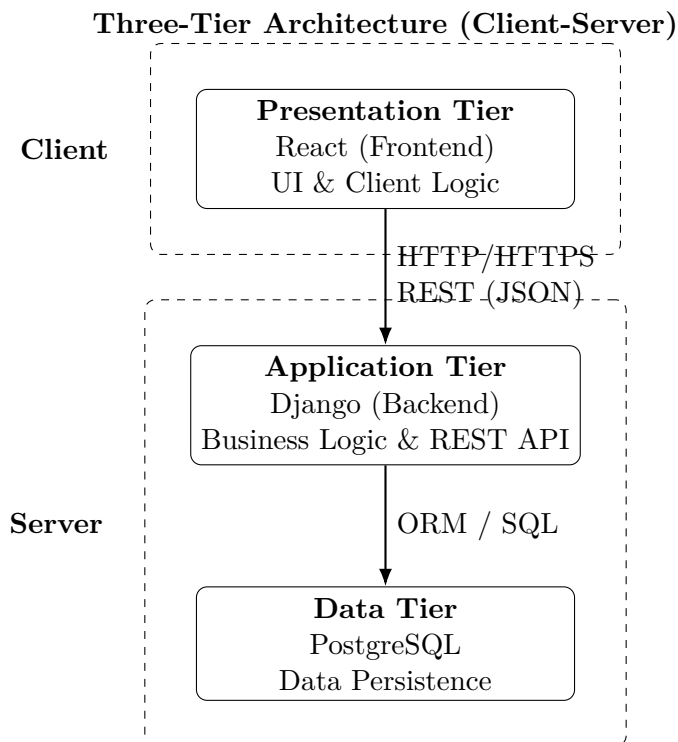


Figura 6.1: Schema architetturale Three-Tier con separazione Client/Server.

6.1 Presentation Tier

Il **Presentation Tier** è implementato mediante **React**, che costituisce il client dell'applicazione e si occupa della gestione dell'interfaccia utente e dell'interazione con l'utente finale. La comunicazione con il livello applicativo avviene tramite **API RESTful**, utilizzando il protocollo HTTP/HTTPS e lo scambio di dati in formato JSON.

Più precisamente, il frontend del sistema è sviluppato come una **Single Page Application** moderna, basata su React, TypeScript e Vite, ed è responsabile della composizione delle pagine, della navigazione tra le varie sezioni applicative, della gestione dei form e della presentazione dinamica dei dati ricevuti dal backend. Oltre alla semplice visualizzazione delle informazioni, questo livello gestisce anche aspetti più avanzati dell'esperienza utente, come il caching dei dati remoti, l'aggiornamento dell'interfaccia in seguito a operazioni asincrone e la ricezione di eventi realtime, in particolare per notifiche e aggiornamenti relativi alle issue. Il **Presentation Tier** non contiene la logica di business centrale, ma si limita a coordinare l'interazione utente-sistema, delegando al backend tutte le verifiche di autorizzazione, validazione e consistenza applicativa. Questa scelta riduce l'accoppiamento tra interfaccia e dominio applicativo e favorisce una maggiore riusabilità e testabilità del codice frontend.

6.2 Application Tier

L'**Application Tier** è realizzato mediante il framework **Django**, che implementa la logica di business del sistema. Questo livello è responsabile della gestione delle richieste provenienti dal client, dell'elaborazione delle regole applicative, del controllo degli accessi e della validazione dei dati. Inoltre, funge da mediatore tra il **Presentation Tier** e il **Data Tier**.

Nel caso di **BugBoard26**, il backend è sviluppato con **Django** e **Django REST Framework** e rappresenta il nucleo funzionale dell'intero sistema. Esso espone le **API** necessarie al frontend, applica i vincoli di dominio, coordina le operazioni di lettura e scrittura dei dati e centralizza i meccanismi di sicurezza. Dal punto di vista strutturale, il backend adotta un'organizzazione modulare, suddivisa in domini funzionali distinti, come autenticazione, gestione utenti, progetti, issue, notifiche e tag. Questa scomposizione consente di contenere la complessità del sistema pur mantenendo una struttura monolitica, scelta particolarmente adatta al contesto del progetto per semplicità di sviluppo, coerenza transazionale e facilità di manutenzione. Inoltre, il livello applicativo non si limita a rispondere a richieste sincrone, ma supporta anche meccanismi di comunicazione realtime attraverso **Server-Sent Events**, utilizzati per la propagazione di notifiche e aggiornamenti delle issue verso il client. Dal punto di vista della sicurezza, questo tier

gestisce l'autenticazione tramite **JWT**, l'uso dei **refresh token**, la protezione **CSRF** per le operazioni mutative, nonché controlli di autorizzazione basati sui ruoli e sulle relazioni tra utenti, progetti e issue. In questo modo, la logica sensibile rimane confinata nel backend, evitando di affidare al client responsabilità critiche per la correttezza del sistema.

6.3 Data Tier

Il **Data Tier** è costituito da un database relazionale **PostgreSQL**, responsabile della persistenza dei dati. L'accesso al database è gestito esclusivamente dall'**Application Tier**, garantendo l'incapsulamento dei meccanismi di accesso ai dati e prevenendo accessi diretti dal livello di presentazione.

Questo livello assicura la memorizzazione strutturata e consistente delle principali entità del dominio applicativo, tra cui utenti, progetti, membership, issue, assegnazioni, eventi della timeline, notifiche e allegati. La scelta di **PostgreSQL** risulta coerente con la natura relazionale del dominio, caratterizzato da molteplici relazioni tra entità e dalla necessità di preservare integrità referenziale, consistenza transazionale e affidabilità nelle operazioni di aggiornamento concorrente. L'isolamento del **Data Tier** rispetto al client rappresenta un aspetto architetturale fondamentale: il frontend non interagisce mai direttamente con il database, ma accede ai dati esclusivamente tramite il backend, che ne controlla modalità di accesso, visibilità e trasformazione. Tale impostazione consente non solo di rafforzare la sicurezza del sistema, ma anche di astrarre i dettagli implementativi della persistenza, rendendo possibile l'evoluzione delle componenti applicative senza impatti diretti sull'interfaccia utente.

6.4 Vantaggi Architettureali

L'adozione dell'**architettura Three-Tier** garantisce:

- Separazione delle responsabilità (**Separation of Concerns**);
- Modularità e manutenibilità del sistema;
- Scalabilità indipendente dei singoli livelli;
- Indipendenza tecnologica tra frontend, backend e database.

A tali benefici si aggiungono ulteriori vantaggi, in primo luogo, la separazione tra i tre livelli favorisce una migliore organizzazione del lavoro di sviluppo, permettendo di intervenire su interfaccia, logica applicativa o persistenza in maniera più controllata e con un minore rischio di introdurre regressioni trasversali. In secondo luogo, la modularità del backend e la chiara distinzione tra responsabilità di frontend e backend

migliorano la testabilità complessiva del sistema, consentendo l'adozione di strategie di verifica differenziate su componenti UI, API, logica di dominio e persistenza. Dal punto di vista evolutivo, l'architettura adottata consente di estendere il sistema con nuove funzionalità mantenendo una base coerente: ad esempio, l'introduzione di nuovi moduli applicativi o di ulteriori endpoint REST può avvenire senza compromettere l'organizzazione generale del progetto. Infine, l'indipendenza tra livelli permette anche una maggiore flessibilità infrastrutturale, come dimostrato dall'uso di Docker Compose per l'orchestrazione dell'ambiente e dall'impiego di Nginx come reverse proxy in produzione.

Capitolo 7

Design del Software e artefatti

7.1 Introduzione

In questo capitolo vengono descritti il design del software di *BugBoard26*, il processo di sviluppo adottato e i principali artefatti prodotti durante la realizzazione del sistema.

L'obiettivo è illustrare la struttura tecnica dell'applicazione, le scelte progettuali che ne hanno guidato lo sviluppo e gli strumenti utilizzati per supportare implementazione, versionamento e controllo della qualità.

In particolare, vengono presentati lo schema di persistenza dei dati, il diagramma delle classi di design, le principali motivazioni architetturali, le evidenze dell'uso di strumenti di versioning, i report di qualità del codice e gli artefatti software sviluppati.

BugBoard26 è stato realizzato come **applicazione web full-stack**, con **frontend** in **React** e **TypeScript**, **backend** basato su **Django** e **Django REST Framework**, **database PostgreSQL** e gestione dell'ambiente tramite **Docker Compose**.

7.2 Struttura e scelte di design

7.2.1 Architettura generale del sistema

Il sistema è stato progettato in modo da mantenere separate le principali responsabilità applicative. Il frontend gestisce la presentazione dei dati e l'interazione con l'utente, mentre il backend centralizza la logica di business, il controllo degli accessi, la persistenza dei dati e l'esposizione delle API.

Questa impostazione consente di ottenere una migliore organizzazione del progetto, facilitando la manutenzione del codice, il testing e l'evoluzione futura del sistema. Anche la struttura del repository riflette questa separazione, distinguendo chiaramente i moduli

applicativi, i file di configurazione, la documentazione tecnica, i test e gli artefatti infrastrutturali.

Accanto ai tre livelli principali, il sistema integra anche alcuni servizi di supporto esterni, utilizzati per completare funzionalità specifiche dell'applicazione in **ambiente di produzione**. Tra questi rientrano **Brevo**, adottato come **provider per le email transazionali**, e i **servizi Google Cloud** utilizzati a **supporto del deployment** e della **gestione dei contenuti multimediali**.

7.2.2 Infrastruttura di deployment su Google Cloud

Dal punto di vista infrastrutturale, il sistema è stato predisposto per essere eseguito in un **ambiente cloud basato su servizi Google**. In particolare, l'esecuzione dell'applicazione può essere ospitata su una **macchina virtuale Google Cloud VM**, utilizzata come nodo di deploy del sistema e come ambiente in cui eseguire i container applicativi.

L'adozione di una macchina virtuale dedicata rappresenta una scelta progettuale utile perché consente di mantenere il pieno controllo dell'ambiente di esecuzione, della configurazione dei **servizi**, delle **variabili di ambiente**, del **reverse proxy** e dei processi necessari al funzionamento del sistema. In questo modo, l'intera applicazione può essere distribuita in modo coerente con l'architettura definita in fase di sviluppo, mantenendo separati frontend, backend, database e proxy anche in produzione.

Per la gestione delle immagini dei container e degli artefatti di build, l'infrastruttura può fare affidamento su **Google Artifact Registry**. Questo servizio permette di conservare in modo centralizzato e versionato le immagini Docker dell'applicazione, semplificando le operazioni di aggiornamento, distribuzione e rilascio delle nuove versioni. L'utilizzo di un registry dedicato migliora inoltre la tracciabilità degli artefatti software distribuiti e rende più ordinato il processo di deployment.

Accanto a tali componenti, il sistema utilizza anche un **bucket Google Cloud Storage** per la gestione persistente dei media in produzione, come avatar utente e allegati associati alle issue. Questa scelta evita di vincolare la persistenza dei file al file system locale della macchina virtuale, migliorando affidabilità, scalabilità e continuità operativa anche in caso di aggiornamenti o ricreazione dei container.

Nel complesso, **l'integrazione con i servizi Google Cloud** consente di separare in modo più netto il livello applicativo da quello infrastrutturale, rendendo il sistema più robusto, più facilmente distribuibile e più aderente a un modello di deployment moderno basato su container e servizi gestiti.

7.2.3 Schema per la persistenza dei dati

La persistenza dei dati è affidata a **PostgreSQL** ed è organizzata secondo un modello relazionale coerente con il dominio applicativo. Le **entità principali** riguardano utenti, progetti, issue, assegnazioni, commenti, allegati, tag e notifiche.

Il modello dati è stato progettato per supportare in modo chiaro le funzionalità collaborative della piattaforma. In particolare, la distinzione tra appartenenza a un progetto e assegnazione a una issue permette di rappresentare in modo più preciso i ruoli degli utenti e i relativi permessi operativi.

Un ulteriore aspetto importante è la gestione degli eventi associati alle issue. Le modifiche non vengono considerate come semplici aggiornamenti di stato, ma come eventi tracciati, così da garantire una **cronologia leggibile delle attività** e una maggiore auditabilità del sistema.

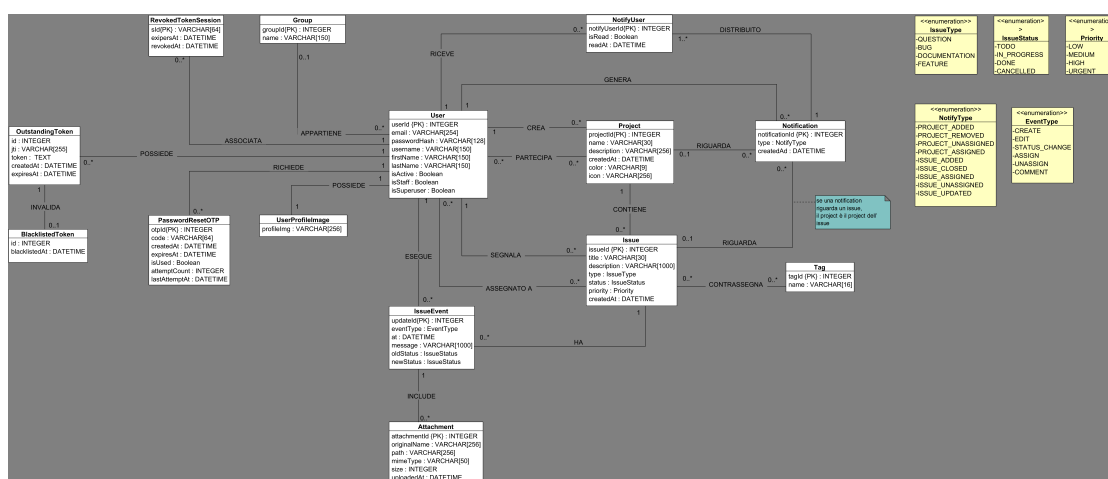


Figura 7.1: Schema della persistenza dei dati di BugBoard26

7.2.4 Diagramma delle classi di design

Il **diagramma delle classi di design** rappresenta le principali entità del sistema e le loro relazioni. Esso consente di visualizzare in modo sintetico la struttura logica del backend e l'organizzazione degli oggetti che implementano il dominio applicativo.

Tra le classi più rilevanti rientrano quelle relative a progetti, membership, issue, assegnatari, eventi, allegati, tag e notifiche. La presenza di tali componenti evidenzia una progettazione orientata alla modularità, nella quale ogni elemento del dominio viene trattato come parte di una struttura coerente e ben definita.

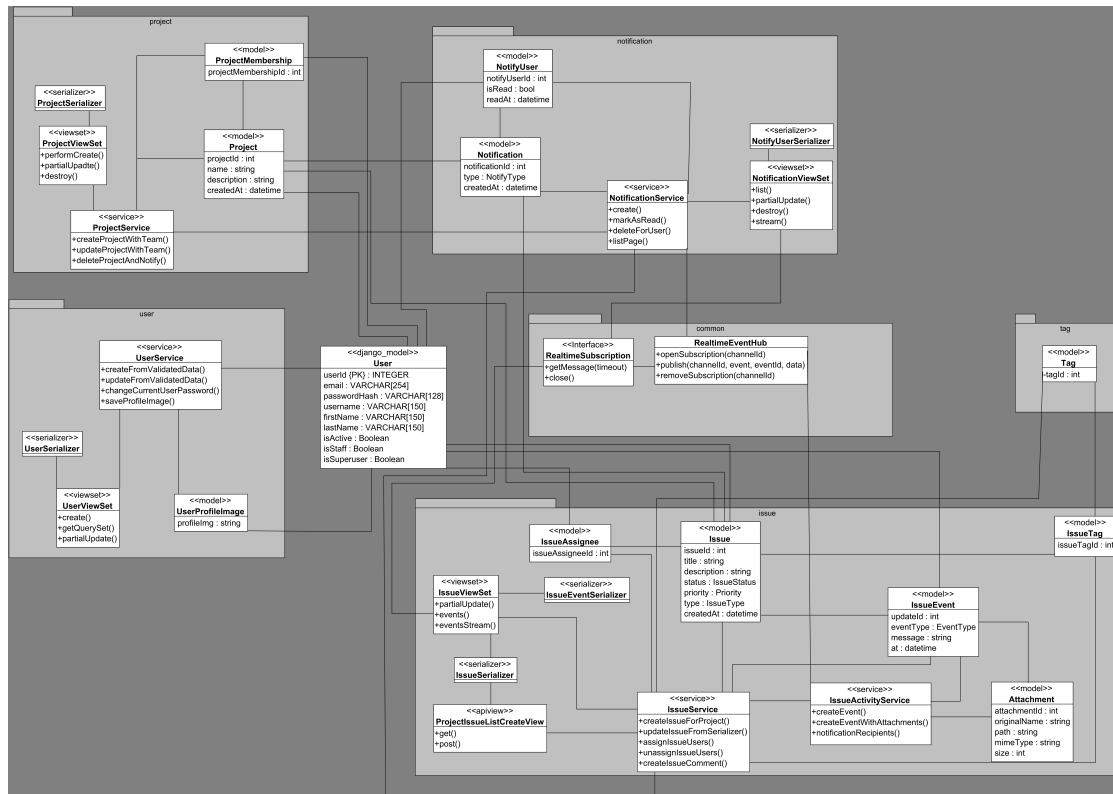


Figura 7.2: Diagramma delle classi di design

7.2.5 Scelte di software design adottate

Le scelte di design adottate nel progetto sono state orientate principalmente a tre obiettivi: chiarezza strutturale, separazione delle responsabilità e facilità di manutenzione.

Una prima scelta significativa è stata l'adozione di una chiara distinzione tra frontend e backend. Tale separazione consente di isolare la logica applicativa dalla presentazione, migliorando la riusabilità del codice e rendendo più semplice l'evoluzione indipendente delle due componenti.

Un secondo elemento importante è l'organizzazione modulare del backend. Pur trattandosi di un'unica applicazione, il sistema è articolato in moduli distinti che gestiscono autenticazione, utenti, progetti, issue, notifiche e tag. Questa soluzione permette di mantenere ordine nel codice senza introdurre la complessità infrastrutturale di un'architettura a microservizi.

Dal lato frontend, la suddivisione in feature, componenti condivisi e layer logici favorisce la riusabilità e il controllo della complessità dell'interfaccia. La gestione dello stato e delle chiamate verso il backend è stata pensata per mantenere coerenza tra i dati visualizzati e lo stato reale del sistema.

Particolare attenzione è stata riservata anche agli aspetti di sicurezza. La gestione dell'autenticazione, dei token e delle autorizzazioni è stata progettata come parte integrante

del sistema, e non come elemento accessorio. Allo stesso modo, la scelta di modellare le modifiche alle issue tramite eventi espliciti consente di avere una cronologia dettagliata delle operazioni svolte dagli utenti.

Infine, per la componente realtime, è stata adottata una soluzione coerente con le esigenze applicative, sufficiente a notificare aggiornamenti lato client senza introdurre meccanismi eccessivamente complessi.

7.2.6 Uso di Brevo per i flussi email

Un ulteriore elemento progettuale rilevante riguarda **la gestione delle email transazionali**. In ambiente di produzione il sistema utilizza **Brevo** come provider per l'invio automatico di messaggi email verso gli utenti.

Tale integrazione è impiegata in particolare nei flussi di sicurezza e onboarding degli account. Da un lato, Brevo supporta il recupero della password mediante **invio di email contenenti il codice OTP** necessario alla verifica dell'identità dell'utente; dall'altro, viene utilizzato anche per i **messaggi associati alla creazione o attivazione di nuovi account**, consentendo al sistema di comunicare in modo affidabile con gli utenti nelle fasi iniziali di accesso alla piattaforma.

L'adozione di un provider esterno dedicato per le email transazionali rappresenta una scelta progettuale utile perché separa la logica applicativa dalla gestione tecnica della consegna delle email, migliorando affidabilità, manutenibilità e controllo del flusso di comunicazione verso l'utente finale.

7.3 Processo di sviluppo

Lo sviluppo del prodotto è stato supportato da un processo organizzato, basato su versionamento del codice, separazione delle linee di lavoro e controlli automatici di qualità.

Il repository mostra l'utilizzo di branch dedicate, utili a distinguere la versione stabile del software dalle attività di sviluppo e testing. Questa organizzazione ha reso più semplice gestire le modifiche, verificare il codice prima dell'integrazione e mantenere una cronologia ordinata dell'evoluzione del progetto.

Un ruolo importante è stato svolto anche dai **workflow automatici di continuous integration**. La presenza di **pipeline** dedicate al **backend**, al **frontend**, ai **contratti API**, al **quality gate** e al **deploy** evidenzia l'adozione di un processo di sviluppo non limitato alla sola scrittura del codice, ma esteso anche alle attività di verifica e validazione continua.

In tale contesto, l'automazione del deploy può essere collegata anche alla componente infrastrutturale cloud. Le pipeline, oltre a verificare la correttezza del codice, possono essere utilizzate per costruire le immagini applicative, pubblicarle su **Artifact Registry** e renderle disponibili all'ambiente di esecuzione in produzione sulla macchina virtuale. Questo rende il processo di rilascio più ordinato, ripetibile e coerente con un **approccio DevOps**.

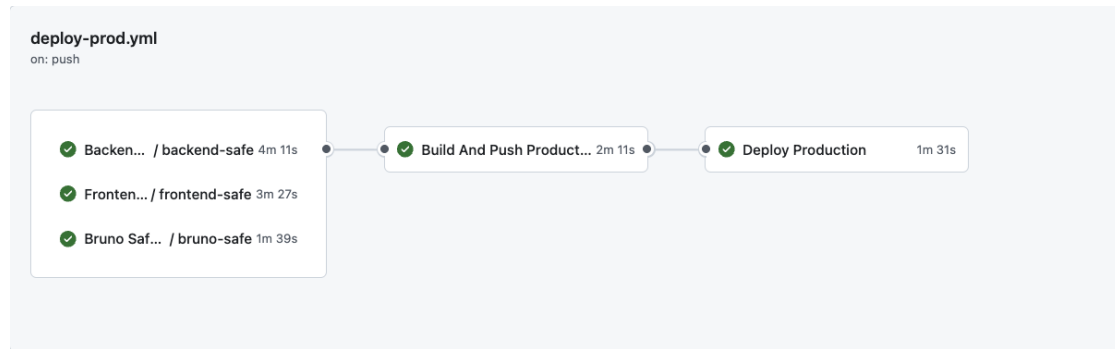


Figura 7.3: Processo di sviluppo e integrazione continua

7.4 Utilizzo di strumenti di versioning

Per la gestione del codice sorgente è stato utilizzato **Git**, con repository ospitato su **GitHub**. L'uso del **versioning** ha permesso di tracciare le modifiche, coordinare il lavoro tra i membri del team e documentare l'evoluzione del progetto nel tempo.

L'analisi del repository consente inoltre di osservare elementi quantitativi utili, come **numero di commit**, **contributori**, **distribuzione delle modifiche** e **frequenza dell'attività di sviluppo**. Tali evidenze rappresentano un indicatore concreto dell'adozione di pratiche collaborative e di una gestione strutturata del codice.

Repository GitHub: <https://github.com/CarmineSgariglia/BugBoard26.git>

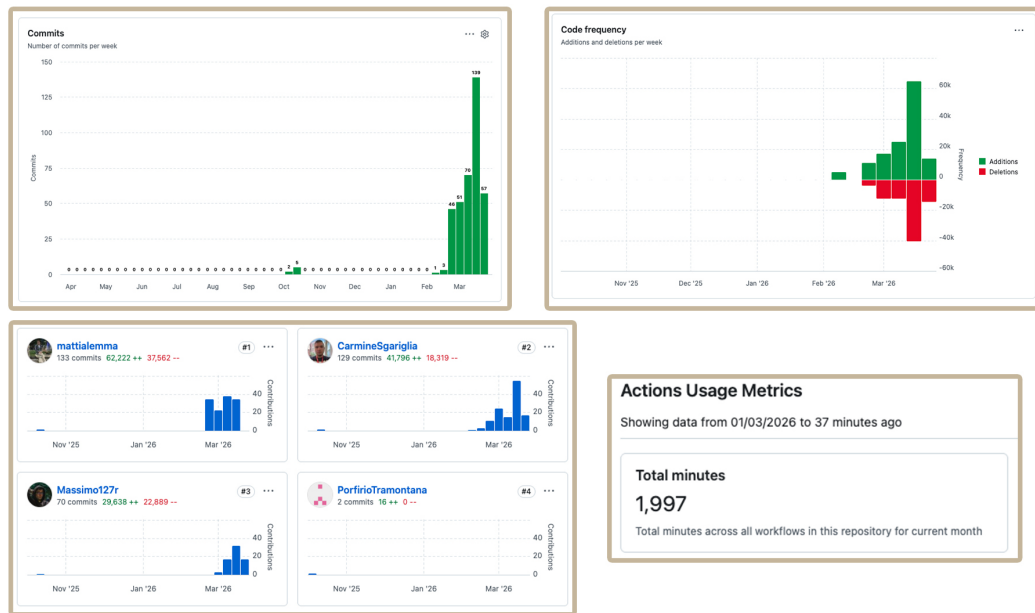


Figura 7.4: Evidenze dell'uso di Git e GitHub

7.5 Report di qualità del codice

La qualità del codice è stata valutata attraverso **strumenti di testing** e **analisi statica**, con particolare attenzione al backend, come richiesto dalla consegna.

I report di qualità consentono di osservare metriche utili relative a copertura del codice, manutenibilità e possibili criticità strutturali. L'impiego di strumenti come **SonarQube** o **SonarCloud** permette di affiancare all'esecuzione dei test anche una verifica continua della qualità interna del software, evidenziando eventuali **code smell**, **duplicazioni** o **vulnerabilità**.

Questa attività risulta particolarmente importante perché consente di supportare con dati oggettivi la robustezza del backend e la qualità complessiva dell'implementazione.

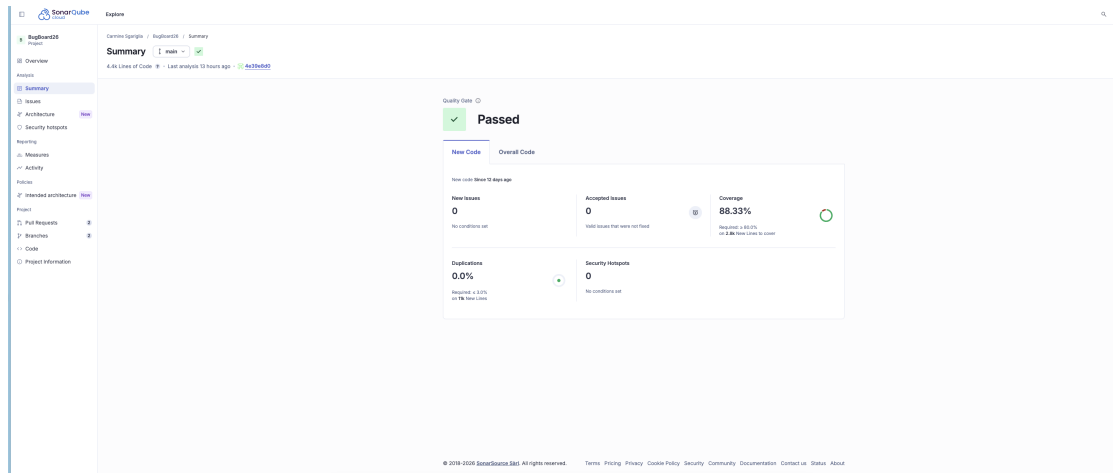


Figura 7.5: Report di qualità del codice

7.6 Codice sorgente sviluppato e artefatti software

Tra gli artefatti prodotti rientrano il **codice sorgente** backend e frontend, i **file Docker** per la containerizzazione, i **file di build automatica**, la **documentazione tecnica delle API**, i **report generati durante le attività di test e validazione**, i **file di mockup** ed i **file di modellazione del dominio** tramite diagrammi.

In particolare, il progetto include i file necessari per l'esecuzione in ambiente locale, per l'integrazione continua e per la configurazione dell'ambiente di produzione. La presenza di **Dockerfile**, file docker-compose e configurazioni dedicate dimostra che il sistema è stato sviluppato considerando anche gli aspetti di esecuzione e distribuzione, e non soltanto la logica applicativa.

Tra gli artefatti infrastrutturali assumono rilievo anche quelli collegati al deployment su cloud. In questo contesto rientrano le immagini container pubblicate su **Artifact Registry**, la configurazione della macchina virtuale usata come nodo di esecuzione del sistema e il **bucket cloud** destinato alla persistenza dei contenuti multimediali. Questi elementi completano il progetto dal punto di vista operativo, poiché rendono possibile il passaggio dal semplice codice sorgente a un sistema effettivamente distribuibile e utilizzabile in produzione.

Accanto al codice sorgente, assumono quindi valore anche tutti gli artefatti complementari che rendono il progetto eseguibile, verificabile e documentabile.

Artifacts			
Produced during runtime			
Name	Size	Digest	
backend-safe-artifacts	301 KB	sha256:225eae55d58495dub4f4f48298775823d088547f63339517251eade7f44de	🔗
bruno-safe-reports	14.9 KB	sha256:16802642d088a76c8488377ae7225d7c2f1875897f7c86d78481c47aa73d	🔗
frontend-safe-artifacts	1.01 MB	sha256:16a79889bc79387c0ba13771b646bc8b648c589f45dc3ba37e255555cc4d7f	🔗

Figura 7.6: Artefatti software del progetto

Capitolo 8

Attività di Testing

8.1 Introduzione

Il presente capitolo descrive l'attività di testing svolta su una porzione selezionata del backend di **BugBoard26**. In particolare, vengono presentati il test plan relativo a una funzionalità scelta dai contraenti, i test di unità automatici sviluppati per due metodi non banali e la documentazione delle strategie adottate per la progettazione dei casi di test.

Per la presente attività è stata predisposta una suite di test dedicata, separata dalla suite automatica standard del progetto, al fine di evitare interferenze con i controlli già presenti nel repository.

Tale suite è collocata nel percorso `backend/unit_test` ed è eseguibile manualmente.

La funzionalità principale scelta per il test plan è il **recupero password tramite OTP**. Tale scelta è motivata dal fatto che il comportamento del sistema, in questo flusso, dipende da molteplici input e da differenti stati iniziali osservabili, tra cui l'utente associato al tentativo, il codice OTP inserito, la presenza o assenza di un OTP aperto, la validità temporale del token e il numero di tentativi errati già effettuati. A questa funzionalità è stato affiancato un secondo metodo, `validate_message()`, appartenente all'area della validazione server-side dei messaggi.

8.2 Test plan per una funzionalità a scelta dei contraenti

8.2.1 Funzionalità selezionata

La funzionalità oggetto del piano di test è il **password reset via OTP**. Si tratta di un flusso significativo dal punto di vista applicativo, poiché il suo comportamento dipende non da un singolo input isolato, ma dalla combinazione tra input forniti dall'utente e stato

iniziale del sistema. In particolare, sono rilevanti:

- l'utente associato al tentativo;
- il codice OTP inserito;
- la configurazione dell'operazione;
- la presenza o assenza di un OTP aperto;
- la validità temporale dell'OTP;
- il numero di tentativi errati già effettuati.

8.2.2 Obiettivi del test plan

Il piano di test definito per la funzionalità OTP persegue i seguenti obiettivi:

- verificare il comportamento del sistema quando non esiste alcun OTP aperto per l'utente;
- verificare il comportamento quando l'OTP più recente esiste ma non è più valido o utilizzabile;
- verificare il corretto riconoscimento di un codice OTP valido;
- verificare il comportamento in caso di match corretto con consumo dell'OTP richiesto;
- verificare il comportamento in caso di codice errato con tentativi ancora sotto soglia;
- verificare il blocco dell'OTP al raggiungimento della soglia massima di tentativi errati.

Per il secondo metodo selezionato, `validate_message()`, l'obiettivo del testing è verificare la correttezza della validazione del messaggio in funzione della presenza o assenza del contenuto, dell'obbligatorietà del campo, della normalizzazione tramite `strip` e del vincolo di lunghezza massima.

8.2.3 Ambito del piano di test

Nel piano di test rientrano i due metodi:

- `_validate_otp_attempt()`
- `validate_message()`

Non rientrano invece nell'ambito di questo capitolo:

- i test end-to-end completi del flusso di recupero password;
- i test di integrazione con il provider email;

- i test del frontend;
- la suite standard di backend già presente nel progetto.

Questa delimitazione dell'ambito è coerente con la natura dell'attività, che richiede una batteria di test automatici su alcuni metodi scelti dal contraente, senza richiedere la copertura completa dell'intero flusso applicativo.

8.2.4 Criteri di ingresso e di uscita

I criteri di ingresso prevedono:

- disponibilità dell'ambiente backend configurato correttamente;
- utilizzo delle impostazioni di test dedicate;
- presenza delle dipendenze Django richieste;
- disponibilità di dati e condizioni iniziali adeguate per costruire gli scenari OTP e i casi di validazione dei messaggi.

I criteri di uscita prevedono:

- esecuzione corretta dei test della suite;
- copertura delle classi di equivalenza individuate;
- copertura dei principali valori limite rilevanti;
- verifica del comportamento osservabile atteso per i due metodi selezionati.

8.3 Codice per test di unità automatici per due metodi non banali

8.3.1 Metodo 1: `_validate_otp_attempt()`

Il primo metodo selezionato è `_validate_otp_attempt()`, appartenente al flusso di recupero password via OTP. Si tratta di un metodo non banale perché il suo comportamento dipende dalla combinazione di più input e dallo stato iniziale dell'OTP più recente associato all'utente. Gli input considerati nella progettazione dei test sono:

- `user`
- `code`
- `lock`
- `consume_on_match`

- stato iniziale dell'OTP più recente associato all'utente

In questo contesto, lo stato iniziale dell'OTP viene trattato come parte dell'input del caso di test. Questa scelta metodologica consente di analizzare il metodo senza fare riferimento a dettagli interni di implementazione, mantenendo quindi la coerenza con un'impostazione interamente black-box.

I casi di test implementati per questo metodo sono i seguenti:

- `test_returns_false_and_none_when_no_open_otp_exists`
- `test_returns_false_and_none_when_latest_otp_is_not_valid`
- `test_returns_true_and_matching_otp_when_code_is_correct`
- `test_consumes_matching_otp_when_requested_and_consume_succeeds`
- `test_wrong_code_increments_attempt_count_and_updates_last_attempt_at`
- `test_wrong_code_locks_otp_after_reaching_max_attempts`

Tali test coprono i principali comportamenti osservabili del metodo rispetto alle diverse classi di input individuate.

8.3.2 Metodo 2: `validate_message()`

Il secondo metodo selezionato è `validate_message()`, relativo alla validazione server-side di messaggi testuali. Anche questo metodo è non banale, poiché il suo comportamento dipende dalla combinazione tra contenuto del messaggio, obbligatorietà del campo e normalizzazione degli spazi.

Gli input considerati sono:

- `message`
- `required`
- `strip`

I casi di test implementati per questo metodo sono:

- `test_returns_empty_string_when_message_is_none_and_not_required`
- `test_preserves_external_spaces_when_strip_is_disabled`
- `test_strips_external_spaces_when_strip_is_enabled`
- `test_rejects_blank_message_when_required_after_strip`
- `test_accepts_message_at_exact_max_length`
- `test_rejects_message_longer_than_max_length`

- `test_converts_non_string_input_before_validation`

Anche in questo caso i test coprono un insieme significativo di comportamenti osservabili, senza dipendere dalla struttura interna del metodo.

8.4 Documentazione delle strategie adottate per la progettazione dei test

8.4.1 Approccio metodologico generale

L'approccio adottato nella progettazione della suite è **interamente black-box**. I casi di test sono stati costruiti sulla base di:

- classi di equivalenza degli input;
- valori limite;
- risultati osservabili in uscita;
- stato iniziale osservabile del sistema.

Questa scelta metodologica consente di motivare i test in funzione degli input e dei risultati attesi, senza introdurre dipendenza dai dettagli interni di implementazione. In particolare, per il metodo OTP, anche lo stato iniziale del record viene trattato come parte dell'input osservabile del caso di test.

8.4.2 Classi di equivalenza per `_validate_otp_attempt()`

Per il primo metodo sono state individuate sei classi di equivalenza:

- **OTP-EC1:** nessun OTP aperto per l'utente;
- **OTP-EC2:** OTP presente ma non valido o scaduto;
- **OTP-EC3:** OTP valido e codice corretto con `consume_on_match=False`;
- **OTP-EC4:** OTP valido e codice corretto con `consume_on_match=True`;
- **OTP-EC5:** OTP valido ma codice errato con tentativi sotto soglia;
- **OTP-EC6:** OTP valido ma codice errato a soglia massima.

A ciascuna di queste classi corrisponde un comportamento osservabile specifico:

- ritorno di `(False, None)` in assenza di OTP o in presenza di OTP non valido;
- ritorno di `(True, otp)` in caso di codice corretto;

- mantenimento dell'OTP aperto o marcatura come usato in funzione della configurazione dell'operazione;
- incremento del numero di tentativi e, alla soglia massima, blocco dell'OTP.

Le classi individuate risultano coerenti con la natura del metodo e con l'insieme dei casi di test effettivamente implementati.

8.4.3 Nota metodologica sul parametro `lock`

Nel metodo `_validate_otp_attempt()`, il parametro `lock` viene trattato come input di configurazione dell'operazione. Esso non è stato utilizzato per costruire casi di test strutturali, ma soltanto come parte del contesto del metodo, nei casi in cui il risultato osservabile resti coerente con la classe di equivalenza selezionata. Questa scelta conferma la coerenza dell'impostazione black-box adottata nell'intero capitolo.

8.4.4 Classi di equivalenza e valori limite per `validate_message()`

Per il secondo metodo sono state individuate cinque classi di equivalenza e due casi di boundary value analysis:

- **MSG-EC1:** `message=None` con campo non obbligatorio;
- **MSG-EC2:** stringa valida con spazi iniziali e finali, `strip=False`;
- **MSG-EC3:** stringa valida con spazi iniziali e finali, `strip=True`;
- **MSG-EC4:** input blank con `required=True` e `strip=True`;
- **MSG-EC5:** input non stringa ma convertibile con `str(...)`.

Ai fini dell'analisi dei valori limite, sono stati inoltre considerati:

- **MSG-BV1:** stringa di lunghezza esattamente pari a 1000, che deve essere accettata;
- **MSG-BV2:** stringa di lunghezza pari a 1001, che deve produrre errore di validazione.

Questa progettazione è particolarmente coerente con il comportamento del metodo, che è completamente descrivibile in termini di dominio di input e risultati osservabili.

8.4.5 Criteri di copertura adottati

Nel presente elaborato non sono stati adottati criteri di copertura strutturale di tipo white-box, poiché l'approccio metodologico scelto è interamente black-box. La completezza della suite viene quindi motivata in funzione della copertura delle classi di equivalenza e dei valori limite significativi.

Per `_validate_otp_attempt()` la copertura è ottenuta mediante la rappresentazione delle principali combinazioni osservabili tra stato iniziale dell'OTP, correttezza del codice e configurazione dell'operazione.

Per `validate_message()` la copertura è ottenuta mediante il partizionamento del dominio di input e la boundary value analysis sul limite massimo di lunghezza del messaggio.

Capitolo 9

Valutazione dell'usabilità

9.1 Introduzione

Il presente capitolo descrive l'attività di valutazione dell'usabilità svolta su **BugBoard26**, con l'obiettivo di analizzare la qualità dell'interazione offerta dal sistema e di individuare eventuali criticità nell'interfaccia utente.

La valutazione è stata inizialmente condotta attraverso una **ispezione euristica** svolta internamente dal team, utilizzando come riferimento le dieci euristiche di Nielsen. Tale attività ha consentito di raccogliere osservazioni strutturate sui principali flussi dell'applicazione e di consolidare i risultati in una sintesi aggregata utile a evidenziare punti di forza, criticità localizzate e possibili direzioni di miglioramento.

A questa analisi si affianca inoltre una seconda fonte di evidenza, basata su un **test guidato di usabilità con utenti reali**. In questo caso non si tratta di una heuristic evaluation in senso stretto, ma di una lettura euristica dei feedback emersi durante l'esecuzione di task concreti e delle risposte fornite dai partecipanti nel questionario finale. L'integrazione tra osservazione esperta e feedback d'uso consente quindi di ottenere un quadro più ricco e più attendibile della qualità percepita dell'interfaccia.

9.2 Metodo di valutazione adottato

La valutazione dell'usabilità è stata realizzata mediante due prospettive complementari.

La prima consiste in una procedura di **heuristic evaluation**. Ogni componente del team ha compilato individualmente una checklist strutturata basata sulle **dieci euristiche di Nielsen**, analizzando i principali flow dell'applicazione dal punto di vista dell'interazione, della chiarezza dell'interfaccia e della coerenza dei comportamenti osservabili.

Successivamente, le schede raccolte sono state aggregate e consolidate in un unico report,

allo scopo di deduplicare le segnalazioni, uniformare la formulazione delle criticità emerse e ottenere un quadro sintetico ma rappresentativo dei problemi osservati. Questo approccio ha permesso di trasformare osservazioni inizialmente distribuite in una base dati qualitativa più leggibile e utilizzabile ai fini della documentazione finale.

La seconda prospettiva consiste in un **test guidato di usabilità** condotto su un campione di 6 partecipanti, tutti in grado di completare il test in autonomia seguendo le istruzioni fornite in un documento dedicato. Il campione comprendeva sia utenti con familiarità tecnica medio-alta, sia utenti con profilo non tecnico o con esposizione limitata al dominio del bug tracking. Questa eterogeneità ha consentito di osservare sia la tenuta dell'interfaccia rispetto al target naturale del sistema, sia la sua comprensibilità da parte di utenti meno esperti.

Dal punto di vista metodologico, è importante chiarire che la seconda attività non coincide con una heuristic evaluation formale, ma con una lettura interpretativa dei feedback raccolti da utenti reali alla luce di principi di usabilità. I risultati ottenuti, pertanto, non intendono dimostrare l'assenza assoluta di problemi, ma forniscono una misura ragionata della qualità d'uso osservata nei flussi esaminati.

9.3 Raccolta dei dati e contesto del test

Nel test guidato di usabilità, ai partecipanti è stato chiesto di eseguire 5 task sequenziali:

1. aprire il progetto corretto;
2. trovare una issue specifica;
3. creare una nuova issue;
4. assegnare la issue a un developer;
5. aggiungere un commento con allegato.

Per ciascun task, i partecipanti hanno indicato se erano riusciti a completarlo, quanto l'attività fosse risultata facile, quanto si sentissero sicuri di aver svolto correttamente l'azione richiesta e in quale punto avessero eventualmente avuto dubbi. Al termine del test, è stato inoltre somministrato un questionario finale volto a raccogliere una valutazione complessiva dell'interfaccia, della chiarezza dei comandi e della predisposizione a riutilizzare il sistema in futuro.

Un elemento rilevante per la corretta interpretazione dei risultati riguarda il contesto d'uso previsto del prodotto. **BugBoard26** è stato progettato come strumento rivolto a team interni e a utenti con una certa familiarità con processi tecnici, issue e flussi collaborativi di progetto. Per questa ragione, i feedback dei partecipanti meno vicini a tale dominio

sono particolarmente utili per valutare la **learnability** iniziale, ma non devono essere letti come segnale di inadeguatezza del sistema rispetto al suo pubblico di riferimento.

9.4 Sintesi generale dei risultati

L'analisi aggregata dei dati restituisce un quadro complessivamente positivo. In gran parte dei flussi analizzati non sono state rilevate criticità significative, mentre le problematiche emerse risultano limitate, localizzate e di severità bassa o media. In particolare, non sono stati individuati problemi classificati con severità 3 o 4, mentre la distribuzione consolidata mostra due criticità di severità 2 e cinque criticità di severità 1.

Questo risultato suggerisce che, nei principali flow osservati, **BugBoard26** presenta una buona base di usabilità e non manifesta problemi bloccanti o sistemici. Le segnalazioni raccolte non indicano infatti una compromissione generale dell'esperienza d'uso, ma piuttosto la presenza di alcuni punti specifici nei quali l'interfaccia può essere affinata.

Le aree in cui sono emerse meno criticità osservabili durante l'ispezione sono risultate essere **Project**, **Issue**, **Settings** e **Auth**. Anche questo dato va interpretato correttamente: esso non dimostra l'assenza assoluta di problemi in tali moduli, ma indica che, nel corso dell'ispezione effettuata, non sono emerse violazioni evidenti delle euristiche considerate.

I risultati del test guidato con utenti confermano questo quadro generale. Tutti i 6 partecipanti hanno completato con successo tutti i task previsti, senza che emergessero blocchi funzionali gravi. Inoltre, dal questionario finale risulta una valutazione media generale pari a circa **4,1/5**, valore che suggerisce una buona usabilità percepita. La dimensione più solida riguarda la chiarezza del flusso di creazione e modifica di una issue, che raggiunge circa **4,5/5**; risultano invece relativamente più deboli la facilità nel trovare progetti, issue e azioni principali, con una media di circa **3,67/5**, e la qualità dei feedback di sistema dopo azioni, salvataggi o errori, con una media di circa **3,83/5**.

9.5 Distribuzione delle criticità per severità

La distribuzione aggregata per severità mostra un profilo di rischio contenuto. Le criticità individuate si collocano esclusivamente nei livelli più bassi della scala adottata, mentre non risultano problemi di severità elevata.

Tabella 9.1: Distribuzione aggregata delle criticità per severità

Severità	Numero di criticità
4	0
3	0
2	2
1	5

La lettura di questa distribuzione conferma che l'usabilità del sistema, nei flow osservati, non risulta compromessa da errori gravi o bloccanti. Le problematiche emerse hanno invece natura circoscritta e suggeriscono interventi di miglioramento progressivo, più che correzioni strutturali dell'interfaccia.

Anche i dati raccolti durante il test guidato rafforzano questa interpretazione. I task sono stati giudicati generalmente accessibili, con valori medi di facilità percepita compresi tra circa **4,17/5** e **4,83/5**. La sicurezza percepita, invece, risulta più bassa soprattutto nei task iniziali, quelli maggiormente legati all'orientamento nell'interfaccia. Questo dato suggerisce che l'attrito principale si concentri nella fase di ingresso nel sistema, più che nell'esecuzione dei flussi operativi centrali.

9.6 Aree principali in cui sono emerse criticità

Dall'analisi aggregata risulta che le osservazioni si concentrano soprattutto in due aree: alcuni modal, in particolare nel flow di creazione issue, e il sistema delle notifiche globali.

9.6.1 Modal di creazione issue

Una prima area di attenzione riguarda il modal di creazione di una nuova issue. Le criticità consolidate evidenziano principalmente due aspetti.

Il primo riguarda la presenza di elementi di chiusura o uscita percepiti come ridondanti o non pienamente chiari. Tale rilievo è stato associato all'euristica relativa al controllo e alla libertà dell'utente, poiché una ridondanza o una debolezza nei controlli di uscita può generare incertezza su come annullare l'operazione o abbandonare il flow.

Il secondo aspetto riguarda la formulazione di alcune etichette, vincoli o indicazioni presenti nel form, percepite come troppo deboli o poco esplicite. In questo caso il problema è stato ricondotto alle euristiche di prevenzione degli errori e di riconoscimento invece di memorizzazione. Una minore esplicitazione dei campi richiesti, dei limiti di compilazione

o dello stato del form può infatti aumentare il carico interpretativo dell'utente e favorire errori evitabili.

Nel complesso, questa area non presenta problemi gravi, ma suggerisce la necessità di rendere il flusso di creazione issue più chiaro, più guidato e meno ambiguo nei meccanismi di compilazione e uscita.

I feedback dei partecipanti al test guidato confermano questa interpretazione: il flusso di creazione issue è stato giudicato complessivamente chiaro, ma alcuni utenti hanno evidenziato un'iniziale incertezza nel capire dove cliccare per avviare correttamente l'azione o come orientarsi nei primi passaggi del task. Ciò suggerisce che il problema principale non sia la funzionalità in sé, bensì la sua immediata riconoscibilità per chi non possiede ancora familiarità con il dominio applicativo.

9.6.2 Sistema di notifiche globali

La seconda area di criticità riguarda il sistema delle notifiche globali, che nel report aggregato emerge come la zona relativamente più sensibile dal punto di vista dell'usabilità. Le osservazioni consolidate indicano da un lato una non piena coerenza tra icone, stati e azioni disponibili, dall'altro una quantità di contesto informativo talvolta insufficiente all'interno della singola notifica.

Il primo problema è stato associato all'euristica di coerenza e standard. Quando naming, stati visivi e comportamento delle azioni non risultano pienamente uniformi, l'utente sperimenta una minore prevedibilità nell'uso del pannello notifiche. Il secondo problema è stato invece collegato all'euristica di riconoscimento invece di memorizzazione, poiché notifiche troppo sintetiche richiedono all'utente di ricordare il contesto precedente per comprenderne il significato.

L'impatto atteso di tali criticità consiste in un aumento del carico cognitivo durante la lettura e l'interpretazione delle notifiche. Di conseguenza, il sistema potrebbe beneficiare di un arricchimento informativo delle notifiche stesse e di una maggiore uniformità negli stati e nelle azioni disponibili.

I dati del test guidato mostrano inoltre un bisogno più generale di migliorare i feedback di sistema dopo azioni, salvataggi o errori. Questo elemento è coerente con le osservazioni raccolte sul sistema notifiche e suggerisce che un miglioramento della visibilità e della qualità informativa dei feedback possa produrre benefici non solo in questa specifica area, ma più in generale nell'esperienza d'uso del prodotto.

9.7 Ulteriori evidenze emerse dal test guidato

I feedback raccolti dai partecipanti consentono di mettere in luce alcune dinamiche che non emergono con la stessa chiarezza dalla sola ispezione euristica.

Un primo elemento riguarda la **comprensione iniziale e l'onboarding**. Alcuni partecipanti dichiarano di non aver capito subito l'interfaccia o di aver avuto dubbi soprattutto all'inizio. Questo dato indica che il primo impatto con **BugBoard26** richiede un certo adattamento, soprattutto per chi non possiede già un modello mentale vicino a quello di un sistema di issue tracking. La difficoltà principale, quindi, non appare legata all'impossibilità di portare a termine i task, ma al costo cognitivo necessario per comprendere da dove iniziare e quale percorso seguire nelle prime interazioni.

Un secondo elemento riguarda la **navigazione e la reperibilità delle azioni**. La ricerca del progetto corretto e della issue specifica non ha impedito il completamento dei task, ma ha ridotto in più casi la sicurezza percepita. I task iniziali mostrano infatti una confidenza media inferiore rispetto a quelli successivi, segnale che il problema non è l'assenza di funzionalità, bensì la loro immediata riconoscibilità da parte di utenti meno abituati a questo tipo di strumenti.

Un terzo aspetto riguarda la **terminologia e il modello mentale**. Una parte rilevante delle difficoltà sembra derivare dal lessico e dal dominio applicativo. Termini come **issue**, **bug**, assegnazione, timeline e allegati risultano naturali per utenti che abbiano già utilizzato strumenti come Jira, Trello o GitHub Issues, mentre per utenti meno esperti il significato operativo di tali elementi deve essere ricostruito sul momento. Sotto questo profilo, il lessico dell'interfaccia risulta coerente con il target professionale di riferimento, ma può risultare meno trasparente per utenti esterni o occasionali.

Infine, il test guidato ha fatto emergere alcune osservazioni puntuali sui **feedback di sistema e sulla visibilità di alcune azioni**. Ad esempio, un partecipante ha riferito di aver inviato prima il messaggio e poi l'allegato senza comprendere subito il flusso corretto; un altro ha osservato che il controllo per aggiungere un allegato sotto la barra del commento non si nota facilmente; un ulteriore feedback ha segnalato che l'ordinamento di default su **oldest** rende poco evidente la issue appena creata, che finisce in basso nella lista. Si tratta di elementi localizzati, ma utili per orientare miglioramenti concreti nell'esperienza d'uso.

9.8 Interpretazione complessiva dei risultati

La lettura complessiva dei dati raccolti mostra che l'ispezione euristica non ha evidenziato problemi di usabilità diffusi sull'intera applicazione. Le criticità emerse risultano infatti poche, circoscritte e prive di severità alta. Questo consente di affermare che, nei principali

flow osservati, **BugBoard26** appare generalmente utilizzabile e non presenta ostacoli evidenti tali da compromettere l'interazione in modo sostanziale.

Allo stesso tempo, il test guidato con utenti reali aggiunge un'informazione importante: la differenza più significativa non riguarda tanto la possibilità di completare i task, quanto il diverso livello di sicurezza percepita tra utenti più tecnici e utenti meno tecnici. In particolare, gli utenti meno tecnici esprimono un giudizio medio globale di circa **3,71/5** e una confidenza media nei task di circa **3,87/5**, mentre gli utenti più tecnici raggiungono un giudizio medio globale di circa **4,5/5** e una confidenza media nei task di circa **4,8/5**. Questo dato supporta l'ipotesi interpretativa secondo cui le difficoltà emerse riguardano soprattutto contesto, terminologia e orientamento iniziale, più che veri ostacoli operativi interni ai flussi.

Allo stesso tempo, i dati raccolti permettono di individuare alcune direttrici di miglioramento precise. In particolare, dall'aggregazione dei risultati emerge l'opportunità di intervenire su:

- chiarezza e non ridondanza dei controlli nei modal;
- esplicitazione di etichette, vincoli e stato dei form;
- coerenza di icone, stati e azioni nel sistema di notifiche;
- aumento del contesto informativo mostrato nelle notifiche;
- miglioramento dell'onboarding iniziale e della reperibilità delle azioni principali;
- maggiore evidenza visiva per comandi e feedback operativi.

Tali indicazioni non suggeriscono una revisione generale dell'interfaccia, ma piuttosto un miglioramento mirato di alcuni componenti. In altre parole, il valore principale della valutazione svolta non consiste nell'aver evidenziato un problema sistemico, ma nell'aver consentito di localizzare con precisione alcune aree in cui l'esperienza utente può essere ulteriormente raffinata.

9.9 Limiti della valutazione svolta

I risultati ottenuti devono essere interpretati tenendo conto della natura dei metodi adottati. L'ispezione euristica costituisce infatti una forma di valutazione esperta, utile a individuare violazioni osservabili delle euristiche di usabilità, ma non sostitutiva di una validazione empirica completa con utenti reali o potenziali.

Parallelamente, il test guidato con utenti ha coinvolto un campione ridotto, pari a 6 partecipanti, e i risultati devono quindi essere letti in chiave descrittiva e qualitativa, più che statistica. Inoltre, non tutti i partecipanti coincidono con l'utenza principale per

cui **BugBoard26** è stato progettato. Questo elemento è rilevante, perché parte delle difficoltà osservate deriva plausibilmente dalla distanza tra il background dei partecipanti e il dominio del bug tracking, più che da limiti intrinseci dell'interfaccia.

In particolare, l'assenza di criticità rilevate in un determinato modulo non equivale a dimostrare l'assenza assoluta di problemi, ma indica soltanto che, nei task osservati e secondo i criteri adottati, non sono emerse violazioni evidenti. Inoltre, la severità bassa o media delle criticità individuate non annulla la loro rilevanza progettuale, ma suggerisce che esse possano essere affrontate in modo incrementale nelle successive iterazioni del sistema.

9.10 Conclusioni

La valutazione dell'usabilità di **BugBoard26** restituisce un quadro complessivamente positivo. Sia l'ispezione euristica interna sia il test guidato con utenti reali mostrano che i flussi principali dell'applicazione risultano generalmente utilizzabili e che non emergono criticità gravi o bloccanti. Tutti i task previsti nel test con utenti sono stati completati con successo, mentre la valutazione media generale dell'interfaccia risulta positiva.

Le problematiche rilevate risultano circoscritte soprattutto a tre aree: chiarezza iniziale dell'interfaccia, reperibilità delle azioni nei primi task e qualità informativa dei feedback di sistema, inclusi i meccanismi di notifica. Ne consegue che la priorità progettuale non appare quella di una revisione generale dell'applicazione, bensì di un insieme di interventi mirati volti a migliorare onboarding, orientamento iniziale, visibilità dei comandi e qualità del contesto fornito all'utente.

Nel complesso, l'analisi svolta consente di sostenere che **BugBoard26** possiede una base funzionale solida, flussi centrali ben strutturati e una buona aderenza al pubblico di riferimento, pur mantenendo margini di miglioramento localizzati che possono essere affrontati efficacemente nelle successive iterazioni progettuali.



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

Documentazione

Progetto Ingegneria del Software

Carmine Sgariglia (N86005069)

Mattia Lemma (N86005170)

Massimo Russo (N86005016)

Anno Accademico 2025/2026